DATABASE MANAGEMENT SYSTEMS

A Comprehensive Study Guide

BIT Year 2 | 2025

Topic	Coverage
0. Core Database Concepts	DBMS Types & Architecture, Entities (Strong/Weak), Attributes (All Types), Relationships (Degree, Participation), Cardinality (1:1, 1:N, M:N, Crow's Foot)
1. Entity Relationship Model & Attributes	ER Diagrams, Attribute Types, Keys, Relationships
2. Aggregation & ISA Hierarchies	Generalisation, Specialisation, Inheritance, Aggregation
3. Advanced SQL	INDEX, COUNT, ORDER BY, HAVING, DROP, CHECK
4. Normalisation & Denormalisation	1NF – 3NF, BCNF, Denormalisation Strategies
5. Transaction Management	ACID, Concurrency, Locking, Deadlocks, Recovery
6. Data Backup & Recovery	Backup Types, Recovery Models, DBCC, Best Practices
7. Examination Questions	10 scenario-based questions per topic (60 total)

PART 0: Core Database Concepts

Before diving into ER modelling and SQL, it is essential to understand five foundational concepts that underpin every database system: Database Management Systems (DBMS), Entities, Attributes, Relationships, and Cardinality. Mastery of these concepts is the basis for every topic in this course.

0.1 Database Management System (DBMS)

0.1.1 What Is a Database?

A database is an organised, structured collection of related data stored electronically so that it can be easily accessed, managed, and updated. The key word is organised — data in a database is not random; it is arranged according to a defined model (e.g., tables, documents, graphs) to serve specific purposes.

A simple analogy: a school filing cabinet contains folders (tables), each folder holds student record cards (rows), and each card has labelled fields like Name, Date of Birth, and Grade (columns). A database is the digital, high-speed, multi-user equivalent of that filing cabinet.

Term	Analogy (Filing Cabinet)	Database Equivalent
Database	The entire filing cabinet	A named database (e.g., SchoolDB)
Table	A labelled folder inside the cabinet	A structured table (e.g., Student)
Row / Record	A single student's record card	One row in the Student table
Column / Field	A labelled box on the card (Name, DOB)	A column in the table (Name VARCHAR, DOB DATE)
Primary Key	The unique student ID printed on each card	StudentID INT PRIMARY KEY
Index	A thumb-tab alphabetical divider	An index on LastName for fast lookup

0.1.2 What Is a DBMS?

A Database Management System (DBMS) is a software system that provides an interface between end users (or applications) and the database itself. It handles all the complexity of physically storing, retrieving, updating, and securing data — so that users and developers work with data at a logical level without worrying about how bytes are arranged on disk.

The DBMS acts as a gatekeeper and coordinator for every operation on the data:

- Storage Engine: Manages how data is physically written to and read from disk (pages, extents, data files).
- Query Processor: Parses SQL statements, optimises them into execution plans, and retrieves results.
- Transaction Manager: Ensures all changes obey ACID properties — no partial updates, no conflicts between concurrent users.

- Security Manager: Enforces authentication (who can log in) and authorisation (what they can do — SELECT, INSERT, DROP, etc.).
- Concurrency Control: Manages locks and isolation levels so that thousands of users can work simultaneously without corrupting each other's data.
- Backup and Recovery Manager: Maintains transaction logs and coordinates backup/restore operations.

0.1.3 Types of DBMS

Type	Model	How Data Is Stored	Best Used For	Examples
Relational (RDBMS)	Tables with rows and columns; relationships via foreign keys	Normalised tables; B-tree indexes	Structured data, OLTP systems, financial records	MySQL, PostgreSQL, Microsoft SQL Server, Oracle
NoSQL — Document	Documents (JSON/BSON) with flexible schema	Collections of nested key-value documents	Content management, product catalogs, user profiles	MongoDB, CouchDB, Firestore
NoSQL — Key-Value	Simple key → value pairs	Hash maps / in-memory structures	Caching, session storage, shopping carts	Redis, DynamoDB, Memcached
NoSQL — Column-Family	Columns grouped into families; rows identified by row keys	Column-oriented storage	Time-series data, IoT, analytics at petabyte scale	Apache Cassandra, HBase
NoSQL — Graph	Nodes (entities) and edges (relationships) with properties	Graph data structures (adjacency lists)	Social networks, recommendation engines, fraud detection	Neo4j, Amazon Neptune
NewSQL	Relational model + horizontal scalability	Distributed sharded tables	Cloud-native OLTP at global scale	Google Spanner, CockroachDB, TiDB
Object-Oriented	Objects with attributes and methods stored directly	Object store / object-relational mapping	CAD/CAM, multimedia, complex data types	db4o, ObjectDB

0.1.4 DBMS Architecture — The Three-Schema Architecture

The ANSI/SPARC three-schema architecture separates database design into three distinct levels, isolating users from physical storage details:

Level	Also Called	What It Describes	Who Uses It	Example
External Level	View level / User level	How individual users or applications see the	End users, application	A payroll clerk sees only

		data. Each user can have a different view.	developers	EmployeeID, Name, Salary; cannot see medical records
Conceptual Level	Logical level	The logical structure of the entire database — all entities, attributes, relationships, and constraints. Independent of physical storage.	Database designers, DBAs	The full ER diagram and relational schema (all tables and columns)
Internal Level	Physical level	How data is physically stored — file organisation, indexes, data compression, storage allocation.	DBMS engine, storage engineers	B-tree index on EmployeeID; data pages of 8KB stored in .mdf files on SSD

0.1.5 Advantages of a DBMS over Flat Files

Problem with Flat Files	How DBMS Solves It
Data redundancy: same data repeated in multiple files (e.g., customer address in sales file, delivery file, billing file)	Normalised tables store data once; all applications reference the same source of truth
Data inconsistency: updates in one file are not reflected in others — address changed in billing but not delivery	Single update to the Customer table propagates everywhere via foreign keys
No concurrent access: two people editing the same spreadsheet simultaneously corrupt it	DBMS concurrency control with locking and MVCC allows thousands of simultaneous users safely
No security: anyone with file access can read or edit everything	Role-based access control — a payroll clerk gets SELECT on Salary but not UPDATE; a manager gets INSERT but not DROP TABLE
No integrity enforcement: nothing stops entering age = -50 or a sale for a non-existent customer	Constraints (CHECK, FOREIGN KEY, NOT NULL, UNIQUE) enforced at engine level — bad data is rejected
No crash recovery: if computer crashes mid-write, file is corrupted	Write-Ahead Logging ensures that even a mid-write crash can be fully recovered on restart
Difficult querying: must write custom code to search flat files	Declarative SQL: SELECT Name FROM Customer WHERE City = 'Kuala Lumpur' ORDER BY Name — no loops needed

0.1.6 Popular DBMS Comparison

DBMS	Type	Licence	Strengths	Common Use Cases
Microsoft SQL Server	RDBMS	Commercial (free Express edition)	Deep Windows integration; excellent tooling (SSMS); built-in BI tools; strong transaction support	Enterprise ERP, banking, healthcare, government systems

MySQL / MariaDB	RDBMS	Open-source (GPL)	Very fast reads; massive community; easy to set up; excellent web hosting support	Web applications, e-commerce (Lazada, Shopee use MySQL variants)
PostgreSQL	RDBMS	Open-source (BSD)	Advanced SQL features; JSON support; extensible; best standards compliance; strong for geospatial data	Financial systems, analytics platforms, modern web backends
Oracle Database	RDBMS	Commercial	Industry-leading for VLDB; advanced partitioning; RAC for high availability; used by largest enterprises	Telcos, banks (Maybank, CIMB use Oracle), airlines, governments
MongoDB	Document NoSQL	Open-source (SSPL)	Flexible schema; fast development; horizontal scaling; JSON-native	Content platforms, product catalogs, real-time analytics, mobile backends
Redis	Key-Value NoSQL	Open-source (BSD)	Microsecond latency; in-memory; supports data structures beyond simple values	Session caching, rate limiting, leaderboards, real-time messaging queues
SQLite	Embedded RDBMS	Public Domain	Zero configuration; serverless; entire DB in one file; ACID compliant	Mobile apps (Android/iOS), desktop apps, unit testing, IoT devices

0.2 Entities

0.2.1 Definition

An entity is any real-world object, person, place, concept, or event about which data needs to be collected and stored. In the ER model, an entity represents a distinct and identifiable thing that is meaningful to the business or system being modelled.

Three essential questions to identify an entity:

1. Can it be distinguished from other things? (It has an identity.)
2. Do we need to store data about it? (It has attributes we care about.)
3. Can there be multiple instances of it? (e.g., not just 'one student' but a class of students.)

0.2.2 Entity vs. Entity Set vs. Entity Instance

Term	Definition	Example
Entity Type / Entity Set	The category or class — the template describing what data is stored for this type of thing. Shown as a rectangle in ER diagrams.	Student — the concept of a student, with attributes StudentID, Name, and GPA
Entity Instance / Occurrence	One specific member of that entity set — a single real object.	The specific student Alice Tan (StudentID: S001, Name: Alice Tan, GPA: 3.8)
Entity	Often used interchangeably with Entity Type in casual usage. In	Usually means the rectangle in an ER diagram

	formal usage, refers to the type.	

0.2.3 Strong Entities

A strong entity (also called a regular entity) can exist independently and has its own primary key that uniquely identifies each instance without needing to reference any other entity. Strong entities are represented by a single rectangle in ER diagrams.

Strong Entity	Primary Key	Example Attributes	Why It Is Strong
Student	StudentID	Name, DateOfBirth, Email, GPA	A student exists independently — not tied to any other entity for identification
Product	ProductID	Name, Price, Category, StockQty	A product exists regardless of whether it has been sold or ordered
Hospital	HospitalID	Name, Address, PhoneNumber, Capacity	A hospital record exists on its own, not defined by any other entity
Flight	FlightNumber + FlightDate	DepartureCity, ArrivalCity, AircraftType	Composite natural key; still independent of any passenger or booking

0.2.4 Weak Entities

A weak entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity (called its owner entity or identifying entity) for its existence and identification. Weak entities are represented by a double rectangle, and the identifying relationship is shown as a double diamond.

A weak entity has a partial key (sometimes called a discriminator) — an attribute that, combined with the primary key of the owner entity, uniquely identifies the weak entity instance. The partial key is shown with a dashed underline in ER diagrams.

Weak Entity	Owner (Strong) Entity	Partial Key	Full Identifying Key	Why It Is Weak
Dependent	Employee	DependentName	(EmployeeID, DependentName)	Two employees could have dependents both named 'Emma' — only the combination is unique
OrderLine	Order	LineNumber	(OrderID, LineNumber)	Line 1 appears in every order; only (OrderID, LineNumber) is unique across the

				DB
RoomBooking	Hotel	RoomNumber + CheckInDate	(HotelID, RoomNumber, CheckInDate)	Room 101 exists in many hotels; the booking is tied to a specific hotel instance
Chapter	Book	ChapterNumber	(ISBN, ChapterNumber)	Chapter 3 exists in every book; identity requires knowing which book
InstallmentPayment	LoanAccount	InstallmentNumber	(LoanID, InstallmentNumber)	Payment #5 means nothing without knowing which loan it belongs to

0.2.5 Other Special Entity Types

Entity Type	Description	ER Notation	Example
Associative Entity (Junction Entity)	Created to resolve a many-to-many relationship. Represents the relationship itself as an entity when that relationship needs its own attributes.	Rectangle (after converting M:N to two 1:N)	Enrollment(StudentID, CourseID, EnrollmentDate, Grade) — the relationship between Student and Course has its own data
Superclass Entity	A higher-level entity in an ISA hierarchy. Contains attributes common to all its subclasses.	Rectangle at top of IS-A triangle	Person(PersonID, Name, DOB) — the superclass for Student and Teacher
Subclass Entity	A lower-level entity that inherits from its superclass and adds its own specific attributes.	Rectangle below IS-A triangle	Student(RollNo, Programme) — inherits PersonID, Name, DOB from Person
Aggregated Entity	A relationship set treated as an entity to allow it to participate in another relationship.	Box drawn around the relationship diamond	The SPONSORS relationship (Department, Project) treated as an entity so Employees can MONITOR it

0.2.6 Identifying Entities — Practical Guidelines

When analysing a business domain to design a database, use these guidelines to identify entities:

- Look for nouns in the business requirements. Nouns that represent categories of things (not one-off items) are likely entities. E.g., 'The system must store information about customers, products, and orders' → Customer, Product, Order are entities.

- Ask: 'Do we need more than one of this?' A single configuration value is probably just an attribute, not an entity. But 'Address' with multiple sub-components and multiple instances per person is an entity.
- Ask: 'Does this thing have properties of its own?' If a noun has attributes (name, date, cost, status), it deserves to be an entity. If it only has one attribute, it might just be an attribute of something else.
- Ask: 'Does this thing participate in relationships?' An entity typically relates to other entities. If an item has no relationships, question whether it needs to be a separate entity.
- Watch for time-dependent entities. 'Appointment', 'Invoice', 'Session', 'Transaction' are often entities because they represent specific events at specific times with their own data.

0.3 Attributes

0.3.1 Definition

An attribute is a property, characteristic, or fact that describes an entity or a relationship. Attributes hold the actual data values that we store in the database. Every entity must have at least one attribute — at minimum, a key attribute that uniquely identifies it.

In an ER diagram, attributes are shown as ovals connected to their entity by lines. In a relational database, attributes become columns in a table.

0.3.2 Simple (Atomic) Attributes

A simple attribute, also called an atomic attribute, cannot be meaningfully divided into smaller components. It holds a single indivisible value. These map directly to a single column in the relational schema.

Entity	Simple Attribute	Data Type in SQL	Example Value
Employee	EmployeeID	INT PRIMARY KEY	1042
Employee	Salary	DECIMAL(10,2)	7500.00
Product	Price	DECIMAL(8,2)	129.90
Student	GPA	DECIMAL(3,2)	3.75
Customer	DateOfBirth	DATE	1995-07-14
Order	TotalAmount	MONEY	RM 450.00

0.3.3 Composite Attributes

A composite attribute is made up of two or more sub-attributes, each of which can be simple or further composite. The attribute has a meaningful internal structure. Whether to keep it as a composite or store only the atomic sub-attributes depends on whether you need to query or sort by the components individually.

Common composite attributes in real systems:

Composite Attribute	Sub-Attributes	When to Store Composite vs. Atomic
FullName	FirstName, MiddleName, LastName	Always store atomic. Needed for: sorting by surname, personalised greetings (Dear Alice), mail-merge, legal documents.
Address	HouseNo, Street, City, State, PostalCode, Country	Always store atomic for any system that needs: delivery routing, tax jurisdiction lookup, geographic filtering, mapping.
DateTime	Date (year, month, day) + Time (hour, minute, second)	SQL's DATETIME type handles this. But storing Date and Time separately is useful if you need date-only or time-only queries.
PhysicalMeasurements	Height (feet, inches) or (cm), Weight (kg/lbs)	Store in one unit consistently (e.g., always cm). Composite only if the original unit matters (e.g., for aviation weight and balance in lbs).
BankAccountNumber	BankCode + BranchCode + AccountNumber	Store components separately if you need to filter by bank or branch; store combined if always used as a whole (e.g., IBAN).

0.3.4 Single-Valued vs. Multi-Valued Attributes

A single-valued attribute holds exactly one value for each entity instance at any point in time. Most attributes are single-valued.

A multi-valued attribute can hold multiple values simultaneously for one entity instance. Multi-valued attributes are shown with a double oval in ER diagrams. They cannot be directly stored as a column — they require a separate table.

Multi-Valued Attribute	Entity	Why Multi-Valued	Relational Solution
PhoneNumbers	Person/ Customer	A person may have mobile, home, work, emergency contacts — all valid simultaneously	Person_Phone(PersonID [FK], PhoneNumber, PhoneType) — composite PK
EmailAddresses	Employee	Some employees have personal + work emails on file	Employee_Email(EmployeeID [FK], EmailAddress, IsPrimary) — composite PK
Skills	Applicant/ Employee	A developer may know Python, Java, SQL, Docker, and Kubernetes	Employee_Skill(EmployeeID [FK], Skill) — composite PK
Certifications	Doctor/ Engineer	A doctor holds MBBS, MRCP, Fellowship — all valid credentials	Doctor_Cert(DoctorID [FK], Certification, IssuedDate, ExpiryDate)
Languages	Staff	A tour guide speaks Malay, English, Mandarin, Japanese	Staff_Language(StaffID [FK], Language, Proficiency)
PreviousAddresses	Customer	Banking KYC requires address history for past 5	Customer_Address(CustomerID [FK], Address, FromDate,

		years	ToDate)

0.3.5 Stored vs. Derived Attributes

A stored attribute is physically saved in the database. Its value is entered or updated explicitly. A derived attribute is not stored — its value is calculated from one or more other stored attributes at query time. Derived attributes are shown with a dashed oval in ER diagrams.

Derived Attribute	Derived From (Stored Attribute)	SQL Implementation	Why Not Store It?
Age	DateOfBirth	DATEDIFF(YEAR, DateOfBirth, GETDATE())	If stored, it becomes wrong every day unless updated. Deriving it is always accurate.
TotalOrderAmount	UnitPrice × Quantity per line	SUM(UnitPrice * Quantity) in a query or view	Storing it risks inconsistency if UnitPrice or Quantity is later corrected.
YearsOfService	HireDate	DATEDIFF(YEAR, HireDate, GETDATE())	Changes every year — would need daily automated updates if stored.
FullName	FirstName + LastName	FirstName + ' ' + LastName AS FullName	Storing FullName alongside components creates a redundancy (update anomaly).
BMI	Weight (kg) and Height (m)	(Weight / (Height * Height)) AS BMI	Recalculate whenever needed; storing means an update to Weight must also update BMI.
AverageRating	Individual Review Ratings	AVG(Rating) from Reviews table	For small datasets, compute on the fly. For high-traffic, cache in a pre-aggregated table (denormalisation).
AccountBalance	Sum of all deposits minus withdrawals	SUM of TransactionAmount from TransactionLog	Exception: banks DO store balance as a running total for performance, maintained by triggers.

0.3.6 Key Attributes

A key attribute (also called an identifier) is an attribute or set of attributes whose value is unique across all instances of the entity. It is underlined in ER diagrams. Every entity must have a key attribute. Key attributes become the primary key when mapping to a relational schema.

Key Type	Description	Example	Notes
Natural Key	A key that exists naturally in the real world and has business meaning.	PassportNumber, NationalIC, ISBN, VehicleRegistrationNo	Can change over time (name change, registration renewal) — risky as a long-term PK
Surrogate Key	An artificially generated unique identifier with no business meaning — usually an auto-incremented integer or UUID.	CustomerID INT IDENTITY(1,1), OrderID UNIQUEIDENTIFIER	Never changes; stable FK reference; preferred for most database tables
Composite Key	Two or more attributes together form the key — neither alone is unique.	(StudentID, CourseID) for Enrollment; (FlightNo, FlightDate) for Flight	Use when no single attribute uniquely identifies an instance but a combination does
Partial Key	Identifies a weak entity only in conjunction with the owner's PK.	DependentName (with EmployeeID)	Shown as dashed underline in ER diagram; becomes part of composite PK in schema

0.3.7 Attributes of Relationships

Attributes do not only belong to entities — they can also describe relationships. A relationship attribute is a property that describes the association itself, not either of the participating entities.

Relationship	Entities Involved	Relationship Attributes	Why It Belongs to the Relationship (Not an Entity)
Enrollment	Student ↔ Course	EnrollmentDate, Grade, AttendancePercent	EnrollmentDate is when THIS student enrolled in THIS course — it belongs to neither Student nor Course alone
WorksIn	Employee ↔ Department	StartDate, Role, HoursPerWeek	An employee's role in one department differs from their role in another — the attribute describes the specific assignment
Purchases	Customer ↔ Product	PurchaseDate, Quantity,	The discount a specific

		DiscountApplied	customer got on a specific purchase is unique to that transaction
Sponsors	Department ↔ Project	Budget, GrantedDate	The budget is for THIS department's sponsorship of THIS project — it is not a property of either entity alone
Rated	User ↔ Movie	Rating (1–5), ReviewText, ReviewDate	Alice may rate the same movie differently from Bob; the rating belongs to the act of rating, not to Alice or the movie

0.4 Relationships

0.4.1 Definition

A relationship is an association or connection between two or more entity instances. In the ER model, a relationship represents a meaningful business fact: not just that two entities exist, but that they are connected in a specific way that is important to the system.

Relationships are shown as diamonds (rhombuses) in ER diagrams. The diamond is labelled with a verb or verb phrase that describes the nature of the association. Lines connect the diamond to each participating entity.

Relationship	Entity 1	Entity 2	Verb Phrase	What It Means
Works In	Employee	Department	works in	An employee is assigned to a department; a department employs employees
Enrolls In	Student	Course	enrolls in	A student registers for a course; a course is taken by students
Places	Customer	Order	places	A customer creates an order; an order is placed by a customer
Prescribes	Doctor	Medicine	prescribes	A doctor recommends a medicine for a patient — could be ternary with Patient
Manages	Employee	Employee	manages	A self-referencing (recursive/unary) relationship — an employee manages other employees
Sponsors	Department	ResearchProject	sponsors	A department provides funding for a research project
Owns	Customer	BankAccount	owns	A customer holds one or more bank accounts

0.4.2 Degree of a Relationship

The degree of a relationship is the number of entity sets that participate in it.

Degree	Name	Description	Example	How Common?
1	Unary (Recursive)	One entity set related to itself. One instance relates to another instance of the same type.	Employee MANAGES Employee (a manager is also an employee); Course is a PREREQUISITE OF Course	Common for hierarchies, org charts, course sequences
2	Binary	Two different entity sets are related. This is the most common degree in database design.	Student ENROLLS IN Course; Customer PLACES Order; Doctor TREATS Patient	The vast majority of relationships (80–90%) are binary
3	Ternary	Three entity sets all participate in one relationship simultaneously. No two entities alone capture the full meaning.	Supplier SUPPLIES Part TO Department (all three jointly determine the supply arrangement + qty)	Used when three entities are all required to define a meaningful business event
4+	N-ary	Four or more entity sets. Very rare in practice — usually decomposable into lower-degree relationships.	Rare. Most are signs that the design can be simplified.	Avoid unless clearly necessary

0.4.3 Unary (Recursive) Relationships — Deep Dive

A unary relationship is one where an entity is related to itself. This is extremely common in practice and often catches students off guard. Two important sub-types exist:

Sub-Type	Relationship Type	Example	SQL Implementation
Hierarchical (1:N)	One instance relates to many instances of the same type — tree structure	Employee MANAGES Employee: one manager has many direct reports; each employee has at most one manager (ManagerID FK → EmployeeID)	Employee(EmpID [PK], Name, ManagerID [FK → Employee.EmpID]) — NULL for the CEO (no manager)
Network / Peer (M:N)	Many instances relate to many instances — graph structure	Course IS PREREQUISITE OF Course: Math101 is prerequisite of Math201 and Physics101; Math201	CoursePrerequisite(CourseID [PK,FK], PrerequisiteID [PK,FK]) — a bridge

		requires Math101 AND Physics101	table referencing Course twice
Partnership (M:N)	Many instances are associated symmetrically with many others	Employee COLLABORATES WITH Employee (project teams)	EmployeeCollaboration(EmpID1 [PK,FK], EmpID2 [PK,FK]) — ensure EmpID1 < EmpID2 to avoid duplicate pairs

0.4.4 Participation Constraints

Participation constraints (also called existence dependencies) specify whether every instance of an entity must participate in a relationship, or whether participation is optional. There are two types:

Constraint	ER Notation	Meaning	Real-World Example	SQL Implication
Total Participation (Mandatory)	Double line between entity and relationship	EVERY instance of the entity MUST participate in at least one instance of the relationship. An entity without a relationship partner cannot exist.	Every OrderLine MUST belong to an Order. You cannot have an order line without an order.	Foreign key NOT NULL: OrderID INT NOT NULL REFERENCES Order(OrderID)
Partial Participation (Optional)	Single line between entity and relationship	SOME instances may not participate. It is allowed for an entity instance to exist without a relationship partner.	An Employee MAY manage a Department — not every employee is a manager.	Foreign key allows NULL: ManagerID INT NULL REFERENCES Employee(EmpID)

Real-world participation examples across common domains:

Domain	Total Participation (Must Have)	Partial Participation (May Have)
E-commerce	Every OrderLine MUST reference an Order; Every Order MUST reference a Customer	A Customer MAY or MAY NOT have placed any Orders (new registered users)
Hospital	Every Prescription MUST reference a Patient and a Doctor	A Doctor MAY or MAY NOT have any Prescriptions (newly onboarded doctor)
School	Every Enrollment MUST reference both a Student and a Course	A Student MAY or MAY NOT be enrolled in any Course this semester (on leave)
Banking	Every Transaction MUST reference an Account	An Account MAY or MAY NOT have any Transactions (newly opened account)
HR	Every Employee MUST belong to a Department	A Department MAY or MAY NOT have a Manager (some departments are manager-less)

0.4.5 Identifying vs. Non-Identifying Relationships

Type	Definition	Example	ER Notation	SQL Implication
Identifying Relationship	The child entity's identity DEPENDS on the parent — without the parent, the child cannot be identified. Used exclusively with weak entities.	OrderLine depends on Order (an order line has no meaning without its order). Dependent depends on Employee.	Double diamond in ER diagram	Child's PK includes parent's PK as part of composite key: OrderLine(OrderID [PK,FK], LineNo [PK])
Non-Identifying Relationship	The child entity has its own independent identity but is associated with the parent. The child can exist even if the parent is removed (or the FK is set to NULL).	Employee works in Department. Customer places Order.	Single diamond in ER diagram	FK is a regular column (may be nullable): DeptID INT NULL REFERENCES Department(DeptID)

0.5 Cardinality

0.5.1 Definition

Cardinality defines the numerical relationship between entity instances — specifically, how many instances of one entity can be associated with how many instances of another entity through a given relationship. It is one of the most critical design decisions because it determines table structure, foreign key placement, and whether a junction table is needed.

Cardinality is expressed in two complementary ways:

- Maximum Cardinality: The maximum number of instances that can participate (1 or Many).
- Minimum Cardinality: The minimum number that must participate (0 = optional, 1 = mandatory). This is also called participation constraint.

0.5.2 The Three Fundamental Cardinality Ratios

Cardinality	Notation	Meaning	Real-World Analogy
One-to-One (1:1)	1 — 1	One instance of A is associated with at most one instance of B, and vice versa.	A person has one passport; a passport belongs to one person.
One-to-Many (1:N)	1 — N or 1 — *	One instance of A is associated with many instances of B, but each instance of B is associated with only one instance of A.	One mother has many children; each child has one biological mother.
Many-to-Many (M:N)	N — M or * — *	Many instances of A are associated with many instances of B, and vice versa.	Many students take many courses; a course is attended by many students.

0.5.3 One-to-One (1:1) Relationships

In a 1:1 relationship, each entity instance on one side corresponds to at most one entity instance on the other side. These are less common but important when they arise.

Example	Why 1:1?	Relational Implementation	FK Placement Rule
Person — Passport	Each person has at most one valid passport; each passport belongs to exactly one person.	Either: (1) Merge into one table: Person(PersonID, Name, PassportNo), or (2) Passport(PassportNo [PK], PersonID [FK UNIQUE], IssuedDate, ExpiryDate)	FK goes on the table with partial participation, or either side if both are total. Add UNIQUE to ensure 1:1.
Employee — ParkingSpace	Each employee is assigned at most one parking space; each space is assigned to at most one employee.	ParkingSpace(SpaceID [PK], EmployeeID [FK UNIQUE, NULLABLE], Level, SpotNumber)	NULLABLE because not all employees have a space. UNIQUE because a space can only be assigned to one employee.
Country — Capital City	Each country has exactly one capital; each city is capital of exactly one country.	Country(CountryID [PK], Name, CapitalCityID [FK UNIQUE])	FK in Country table. Both sides are total participation (every country must have a capital; every capital must be capital of a country).
User — UserProfile	In a system separating login credentials from profile details for security.	UserProfile(ProfileID [PK], UserID [FK UNIQUE], Bio, AvatarURL, ThemePreference)	FK on UserProfile side — keeps authentication (User) table lean and fast.

0.5.4 One-to-Many (1:N) Relationships

The 1:N relationship is the most common relationship type in relational databases. One entity on the '1' side (parent) can be associated with many entities on the 'N' side (child). The foreign key is always placed in the 'many' side table, pointing back to the '1' side.

Example	The '1' Side	The 'N' Side	FK Placement	Real-World Meaning
Department — Employees	Department (1)	Employee (N)	DeptID in Employee table	One department has many employees; each employee belongs to one department
Customer — Orders	Customer (1)	Order (N)	CustomerID in Orders table	One customer places many orders; each order belongs to one customer
Order — OrderLines	Order (1)	OrderLine (N)	OrderID in OrderLine table	One order has many line items; each line belongs to

				one order
Category — Products	Category (1)	Product (N)	CategoryID in Product table	One category contains many products; each product is in one category
Author — Books (sole author)	Author (1)	Book (N)	AuthorID in Book table	One author writes many books; each book (sole-authored) has one author
Manager — Subordinates	Employee as Manager (1)	Employee as Subordinate (N)	ManagerID (self-FK) in Employee table	Recursive 1:N — one manager supervises many employees

SQL implementation of a 1:N relationship (Customer — Orders):

```
CREATE TABLE Customer (
    CustomerID    INT           PRIMARY KEY,
    FirstName     VARCHAR(50)   NOT NULL,
    LastName      VARCHAR(50)   NOT NULL,
    Email         VARCHAR(100)  UNIQUE NOT NULL
);
```

```
CREATE TABLE Orders (
    OrderID       INT           PRIMARY KEY,
    CustomerID    INT           NOT NULL,         -- FK on the 'many' side
    OrderDate     DATE          NOT NULL,
    TotalAmount   DECIMAL(10,2) NOT NULL CHECK (TotalAmount > 0),
    Status        VARCHAR(20)   DEFAULT 'Pending',
    CONSTRAINT fk_order_customer
        FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
        ON DELETE RESTRICT     -- cannot delete a customer who has orders
        ON UPDATE CASCADE      -- if CustomerID changes, update all orders
);
```

0.5.5 Many-to-Many (M:N) Relationships

A many-to-many relationship cannot be directly implemented in a relational database — there is no clean way to store multiple values in a single column (that would violate 1NF). The standard solution is to introduce a junction table (also called a bridge table, associative table, or cross-reference table) that resolves the M:N into two 1:N relationships.

M:N Example	Junction Table Name	Columns in Junction Table	Additional Relationship Attributes
Student — Course	Enrollment	StudentID [PK,FK], CourseID [PK,FK]	EnrollmentDate, Grade, AttendancePercent
Product — Order	OrderLine	OrderID [PK,FK], ProductID [PK,FK]	Quantity, UnitPrice, DiscountApplied
Doctor — Patient	Consultation	DoctorID [PK,FK], PatientID [PK,FK]	ConsultationDate, Diagnosis, PrescriptionID

Employee — Project	Assignment	EmployeeID [PK,FK], ProjectID [PK,FK]	RoleOnProject, AssignedDate, HoursAllocated
Actor — Movie	ActsIn	ActorID [PK,FK], MovieID [PK,FK]	CharacterName, BillingOrder, ScreenMinutes
Student — Club	Membership	StudentID [PK,FK], ClubID [PK,FK]	JoinDate, MembershipType, IsCommittee

SQL implementation of an M:N relationship (Student — Course via Enrollment):

```
CREATE TABLE Student (
    StudentID   INT          PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    Programme   VARCHAR(50)
);
```

```
CREATE TABLE Course (
    CourseID    VARCHAR(10)  PRIMARY KEY,   -- e.g., 'CS101'
    Title       VARCHAR(100) NOT NULL,
    Credits     INT          CHECK (Credits BETWEEN 1 AND 6)
);
```

```
-- Junction table resolving M:N
CREATE TABLE Enrollment (
    StudentID        INT         NOT NULL,
    CourseID         VARCHAR(10) NOT NULL,
    EnrollmentDate   DATE        NOT NULL DEFAULT GETDATE(),
    Grade            CHAR(2)     CHECK (Grade IN
('A+','A','A-','B+','B','B-',

'C+','C','C-','D','F','I','W')),
    AttendancePercent DECIMAL(5,2) CHECK (AttendancePercent BETWEEN 0 AND
100),
    CONSTRAINT pk_enrollment PRIMARY KEY (StudentID, CourseID),
    CONSTRAINT fk_enr_student FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
    CONSTRAINT fk_enr_course  FOREIGN KEY (CourseID)  REFERENCES
Course(CourseID)
);
```

0.5.6 Min-Max Notation (Crow's Foot Notation)

Beyond the simple 1, N, M notation, database designers often use min-max notation or Crow's Foot notation to specify both the minimum (0 or 1) and maximum (1 or many) participation for each side of a relationship. This is more precise and is widely used in professional database design tools.

Crow's Foot Symbol	Min	Max	Meaning	Example
Exactly One (| |)	1	1	Exactly one —	Each Order MUST have

			mandatory on both min and max	exactly one Customer
Zero or One (O |)	0	1	Optional single — zero or one instance	An Employee MAY have zero or one assigned parking space
One or Many (| <)	1	N	Mandatory many — at least one	Each Order MUST have at least one OrderLine
Zero or Many (O <)	0	N	Optional many — zero, one, or many	A Customer MAY have placed zero or many Orders (new customer has none)
Exactly N (N N)	N	N	Fixed number — rare in practice	A chess match MUST have exactly 2 players

Reading a Crow's Foot diagram: read the symbol closest to the entity, then read outward. For 'Customer — Orders' (Customer side: | O, Orders side: | <): 'A Customer must have zero or many Orders; each Order must have exactly one Customer.'

0.5.7 Complete Cardinality Examples Across Domains

Domain	Relationship	Cardinality	Participation (Min, Max)	Notes
University	Student — Advisor	M:1	Student: (1,1) total; Advisor: (0,N) partial	Every student has exactly one advisor; an advisor may have zero to many students
Healthcare	Patient — Doctor via Consultation	M:N	Both: (0,N) partial	A patient can see many doctors; a doctor can treat many patients; tracked via Consultation junction table
Retail	Invoice — InvoiceLine	1:N	Invoice: (1,1); InvoiceLine: (1,N)	Every invoice has at least one line; each line belongs to exactly one invoice — strong/weak entity pattern
Library	Book — Copy	1:N	Book: (1,1); Copy: (1,N)	One book title has many physical copies; each copy is of one title
Social Media	User — Post	1:N	User: (0,N); Post: (1,1)	A user may have zero posts (new account); each post has exactly one author

Manufacturing	BillOfMaterials	M:N recursive	Component: (0,N); Assembly: (0,N)	A product (assembly) is made of many components; a component can be part of many products — recursive M:N
Transport	Driver — Vehicle	M:N	Both: (0,N) partial	A driver can operate many vehicle types; a vehicle can be driven by many drivers (fleet management)

0.5.8 How Cardinality Determines Schema Design

The cardinality of a relationship directly dictates how it is implemented in a relational schema. This is a hard rule in database design:

Cardinality	Schema Implementation	Where the FK Goes	Junction Table Needed?
1:1	Two tables; add FK with UNIQUE constraint on one side	On the entity with partial participation (optional side) — or either side if both total	No
1:N	Two tables; FK on the 'many' (child) side pointing to the 'one' (parent)	In the 'N' (many) side table — e.g., CustomerID in Orders	No
M:N	THREE tables — two original entities plus a junction/bridge table	Junction table has FKs to both entities; composite PK from both FKs	Yes — mandatory

PART 1: Entity Relationship Model & Attributes

1.1 What Is the ER Model?

The Entity-Relationship (ER) Model, introduced by Peter Chen in 1976, is a conceptual data-modelling technique used to describe the structure of a database at a high level of abstraction. It represents real-world objects (entities), their properties (attributes), and the associations (relationships) between them. The ER model is used during the design phase to create an ER Diagram (ERD) that is later converted into a relational schema.

1.2 Components of the ER Model

Entities

An entity is any real-world object or concept that can be distinctly identified and about which data is stored. Entities are represented as rectangles in ER diagrams.

- Strong Entity: An entity that has its own primary key and exists independently. Example: Student, Employee, Product.
- Weak Entity: An entity that cannot be uniquely identified on its own and depends on a strong entity (its owner entity). Represented with a double rectangle. Example: Dependent (depends on Employee), OrderItem (depends on Order).

1.2.1 Attributes

An attribute is a property or characteristic that describes an entity or a relationship. Attributes are represented as ovals in ER diagrams, connected to their entity by lines.

Attribute Type	Description	ER Symbol	Example
Simple (Atomic)	Cannot be divided into smaller parts	Single oval	Age, EmployeeID, Salary
Composite	Can be divided into sub-attributes	Oval with child ovals	FullName → (FirstName, LastName)
Single-Valued	Holds exactly one value per entity instance	Single oval	DateOfBirth, NationalID
Multi-Valued	Can hold multiple values for one entity	Double oval	PhoneNumbers, Skills
Derived	Value computed from other attributes	Dashed oval	Age (derived from DOB)
Stored	Physically stored in the database	Single oval	DateOfBirth
Key (Identifier)	Uniquely identifies an entity instance	Underlined text in oval	StudentID, SSN
Partial Key	Partially identifies a weak entity (with owner)	Dashed underline	DependentName (with EmpID)

Complex	Composite + multi-valued combined	Nested structure	PreviousAddresses (each with street, city, state)

1.2.2 Composite Attributes – Detailed

A composite attribute is made up of multiple sub-attributes, each of which can be atomic or further composite. The database designer may choose to store only the lowest-level atomic components. For example:

```
Address (Composite)
  ├── Street (Simple)
  ├── City   (Simple)
  ├── State  (Simple)
  └── ZipCode (Simple)
```

When implementing in a relational table, each sub-attribute becomes a separate column: Street, City, State, ZipCode.

1.2.3 Multi-Valued Attributes

A multi-valued attribute must be handled carefully when converting to relational schema. The standard approach is to create a separate table. For example, if Employee has a multi-valued attribute PhoneNumbers:

```
Employee_Phone(EmployeeID  [FK → Employee], PhoneNumber)
```

The primary key of this table is the combination (EmployeeID, PhoneNumber).

1.2.4 Derived Attributes

Derived attributes are not physically stored but are calculated on-the-fly using stored attributes. They reduce redundancy. In SQL, they are often implemented as computed columns or views:

```
-- Derived Age from DateOfBirth
SELECT Name, DATEDIFF(YEAR, DateOfBirth, GETDATE()) AS Age FROM Employee;
```

1.3 Relationships

A relationship represents an association between two or more entities. Relationships are shown as diamonds in ER diagrams. Key properties include:

Property	Types	Description
Degree	Unary, Binary, Ternary	Number of entities participating in the relationship
Cardinality	1:1, 1:N, M:N	Maximum number of instances each entity can participate in
Participation	Total, Partial	Whether all entity instances must participate (total) or only some (partial)

Cardinality Ratios

- One-to-One (1:1): Each instance of Entity A is related to at most one instance of Entity B. Example: Person – Passport.
- One-to-Many (1:N): One instance of A can be related to many instances of B. Example: Department – Employees.
- Many-to-Many (M:N): Many instances of A can relate to many instances of B. Example: Students – Courses. Implemented via a junction/bridge table.

Participation Constraints

- Total Participation (Mandatory): Every entity instance must participate in at least one relationship instance. Shown as a double line. Example: Every Employee must belong to a Department.
- Partial Participation (Optional): Some entity instances may not participate. Shown as a single line. Example: An Employee may or may not manage a Department.

1.4 Keys

Keys are fundamental to the relational model and ER design:

Key Type	Definition	Example
Super Key	Any set of attributes that can uniquely identify a tuple	{StudentID}, {StudentID, Name}
Candidate Key	A minimal super key (no redundant attributes)	{StudentID}, {Email}
Primary Key	The chosen candidate key for a table	StudentID (underlined in ER diagram)
Alternate Key	Candidate keys not chosen as primary key	Email (if StudentID is PK)
Foreign Key	An attribute referencing the PK of another table	DeptID in Employee references Department
Composite Key	A PK made of two or more attributes	(StudentID, CourseID) in Enrollment
Partial Key	Identifies a weak entity together with owner PK	DependentName + EmployeeID

1.5 Converting ER to Relational Schema

Rules for Conversion

4. Each strong entity → one table; PK becomes the primary key.
5. Each composite attribute → separate columns for sub-attributes.
6. Multi-valued attribute → separate table with FK to owner entity.
7. Weak entity → table with FK to owner; PK = owner PK + partial key.
8. 1:1 relationship → place FK on the partial-participation side.

9. 1:N relationship → place FK on the 'many' side.
10. M:N relationship → create junction table with both PKs as composite PK.

🌐 REAL-WORLD EXAMPLE: Hospital Management System — ER Model & Attributes

Scenario: A private hospital needs a database to manage patients, doctors, appointments, rooms, and medical records. Below is a complete ER design demonstrating all attribute types and relationship cardinalities.

ENTITIES AND THEIR ATTRIBUTES:

▶ Patient (Strong Entity)

```
PatientID          → Simple, Key attribute (underlined in ER)
FullName           → Composite: FirstName + LastName + MiddleInitial
DateOfBirth        → Simple, Stored
Age                → Derived (calculated from DateOfBirth)
PhoneNumbers       → Multi-valued (a patient may have home, mobile, emergency)
BloodType          → Simple, Single-valued
Address            → Composite: Street, City, State, PostalCode
```

▶ Doctor (Strong Entity)

```
DoctorID           → Simple, Key
FullName           → Composite
Specialisation     → Simple (e.g., Cardiology, Oncology)
Qualifications     → Multi-valued (MBBS, FRCS, PhD, etc.)
YearsOfExperience  → Derived (from HireDate)
```

▶ Appointment (Strong Entity — relationship with attributes)

```
AppointmentID      → Simple, Key
DateTime           → Simple
Status             → Simple (Scheduled / Completed / Cancelled)
Notes              → Simple (doctor's consultation notes)
```

▶ Room (Strong Entity)

```
RoomNumber         → Simple, Key
Type               → Simple (ICU / General / Private / Theatre)
DailyRate          → Simple
```

▶ Admission (Weak Entity — depends on Patient)

```
AdmissionDate      → Partial Key (combined with PatientID)
DischargeDate      → Simple
Diagnosis          → Simple
```

RELATIONSHIPS AND CARDINALITIES:

Relationship	Entities	Cardinality	Participation
Treats	Doctor — Patient	M:N (a doctor treats many patients; a patient can see many	Partial both sides

		doctors)	
Schedules	Patient — Appointment	1:N (one patient has many appointments)	Total on Appointment side
Conducts	Doctor — Appointment	1:N (one doctor handles many appointments)	Partial on Doctor side
Assigned to	Patient — Room	M:N (a patient may use multiple rooms over stays)	Partial both sides
Has Admission	Patient — Admission	1:N (one patient, many admissions) — Weak entity	Total on Admission side

RELATIONAL SCHEMA (converted from ER):

```
Patient(PatientID [PK], FirstName, LastName, MiddleInitial, DOB, BloodType,
        Street, City, State, PostalCode)

Patient_Phone(PatientID [PK,FK], PhoneNumber [PK], PhoneType)

Doctor(DoctorID [PK], FirstName, LastName, Specialisation, HireDate)

Doctor_Qualification(DoctorID [PK,FK], Qualification [PK])

Appointment(AppointmentID [PK], PatientID [FK], DoctorID [FK],
            DateTime, Status, Notes)

Room(RoomNumber [PK], Type, DailyRate)

Admission(PatientID [PK,FK], AdmissionDate [PK], DischargeDate,
          Diagnosis, RoomNumber [FK])
```

Note: Age is NOT stored — it is computed in queries: DATEDIFF(YEAR, DOB, GETDATE()). YearsOfExperience is similarly derived from HireDate. PhoneNumbers and Qualifications become separate tables due to their multi-valued nature.

PART 2: Aggregation and ISA Hierarchies

2.1 ISA Hierarchies

2.1.1 Generalisation

Generalisation is a bottom-up process in which two or more lower-level entity sets are combined into a single higher-level entity set based on their common attributes. The higher-level entity is called the superclass and the lower-level entities are subclasses.

Example from the notes: Pigeon, Sparrow, and Dove are all generalised into the entity 'Birds'. The entity Birds holds the attributes common to all three. The direction of generalisation is upward in the hierarchy.

2.1.2 Specialisation

Specialisation is the reverse — a top-down process where a higher-level entity is divided into more specific sub-groups based on distinguishing characteristics. The IS-A (IS A) relationship symbol (a triangle pointing upward to the superclass) is used in ER diagrams.

Example: A Person entity can be specialised into Student and Teacher based on their role in a school. Each sub-entity inherits the attributes of Person (Name, Age, Gender) and adds its own specific attributes (RollNo for Student; EmpID for Teacher).

Aspect	Generalisation	Specialisation
Direction	Bottom-up (specific → general)	Top-down (general → specific)
Process	Combining shared attributes of multiple entities	Identifying distinguishing attributes in a group
Result	A superclass entity	One or more subclass entities
Example	Pigeon + Sparrow + Dove → Birds	Person → Student, Teacher, Staff
Trigger	Designer notices common properties	Designer notices unique properties

2.1.3 Inheritance

Inheritance is a key feature of ISA hierarchies. Subclasses automatically inherit all attributes and relationships of their superclass. This avoids data redundancy and supports the object-oriented principle of reuse.

- Person has: Name, Age, Gender
- Student inherits: Name, Age, Gender → and adds: RollNo
- Teacher inherits: Name, Age, Gender → and adds: EmpID

In the relational model, ISA hierarchies can be implemented in three ways:

11. Single Table (Table-per-Hierarchy): One table for all subclasses with a discriminator column. Simple but may have many NULLs.
12. Table per Subclass (Table-per-Type): One table for the superclass and one table per subclass. No NULLs; requires JOINs to retrieve full records.

13. Table per Concrete Class: One table per subclass, duplicating superclass columns. No JOINs; data duplication.

2.1.4 Constraints on ISA Hierarchies

Constraint	Meaning	Example
Disjoint (d)	An entity can belong to only ONE subclass at a time	A Person is either a Student OR a Teacher, not both
Overlapping (o)	An entity can belong to MORE than one subclass simultaneously	A Person can be both a Student and a Part-time Teacher
Total (double line)	Every superclass instance MUST belong to a subclass	Every Vehicle must be a Car, Truck, or Motorcycle
Partial (single line)	A superclass instance may or may not belong to a subclass	Not every Person is a Student or Teacher

2.2 Aggregation

2.2.1 Concept

Aggregation is an ER modelling construct that allows a relationship to be treated as if it were a higher-level entity. It is used when we need to model a relationship involving another relationship set — something standard ER notation cannot express directly.

According to the course textbook (Ramakrishnan & Gehrke, 3rd ed.), aggregation is used when we need to model a relationship involving (entity sets and) a relationship set. Aggregation allows a relationship set to be treated as an entity set for the purpose of participation in other relationships.

2.2.2 When to Use Aggregation

Consider the classic example from the notes: Projects, Departments, and Employees. A Department Sponsors a Project, and an Employee Monitors that sponsorship. The Monitors relationship cannot be directly connected to the Sponsors relationship without aggregation.

Solution: Treat the entire Sponsors relationship (between Projects and Departments) as an aggregated entity. The Monitors relationship then connects Employee to this aggregate. The descriptive attribute 'until' belongs to Monitors.

2.2.3 Aggregation vs. Ternary Relationship

Aggregation and ternary relationships are often confused. The key distinctions are:

Criterion	Aggregation	Ternary Relationship
When to use	When a relationship has its own relationship with another entity	When three entities jointly participate in one association
Descriptive attributes	Belong to the monitoring relationship (e.g., 'until')	Belong to the ternary relationship (e.g., 'qty' in Contracts)
Example	Employee MONITORS (Department SPONSORS Project)	Supplier CONTRACTS Department for Parts with qty
Can substitute?	A ternary cannot always replace aggregation correctly	Three binaries cannot always replace a ternary correctly

2.2.4 Design Decisions in the ER Model

When designing an ER model, several key questions must be answered:

- Entity vs. Attribute: Should Address be an attribute of Employee, or a separate entity? If multiple addresses are needed, or if the structure matters (e.g., query by city), Address must be an entity.
- Entity vs. Relationship: Should 'manages' be a relationship, or should 'Manager' be a sub-entity? If the manager gets a discretionary budget that covers all departments they manage, a Manager sub-entity (using ISA) avoids redundancy.
- Binary vs. Ternary: If a policy is owned by one employee and a dependent is tied to a specific policy, two binary relationships are more accurate than one ternary.
- Aggregation vs. Ternary: Use aggregation when one of the participants in a three-way association is itself a binary relationship with its own descriptive attributes.

🌐 REAL-WORLD EXAMPLE: National University System — ISA Hierarchies & Aggregation

Scenario: A national university needs to model its personnel, research activities, and administrative oversight. This example demonstrates ISA specialisation, inheritance, overlapping constraints, and aggregation in a single integrated design.

PART A — ISA HIERARCHY: University Personnel

The superclass Person captures attributes shared by all individuals at the university:

```
Person (superclass):  PersonID [PK], FirstName, LastName, Email, PhoneNumber,
DateOfBirth
```

Specialisation — OVERLAPPING constraint (a person CAN be both Student and Staff simultaneously, e.g., a PhD candidate who also teaches):

```
Student (subclass):   StudentID, EnrollmentDate, Programme, GPA
                      Inherits: PersonID, FirstName, LastName, Email, PhoneNumber,
DateOfBirth

Staff (subclass):     EmployeeID, Department, Position, Salary, HireDate
                      Inherits: PersonID, FirstName, LastName, Email, PhoneNumber,
DateOfBirth

Lecturer (subclass of Staff):  StaffID [FK], TeachingLoad, OfficeRoom
                      Inherits everything from Staff and Person
```

Real-world consequence of OVERLAPPING constraint: A PhD student (PersonID 1042) who also teaches tutorials has a row in BOTH Student and Staff tables. Queries joining Person + Student + Staff will correctly return all their data.

Relational Schema (Table-per-Subclass implementation):

```
Person(PersonID [PK], FirstName, LastName, Email, PhoneNumber, DOB)
Student(PersonID [PK,FK→Person], StudentID UNIQUE, EnrollmentDate, Programme, GPA)
```

```
Staff(PersonID [PK,FK→Person], EmployeeID UNIQUE, Department, Position, Salary,
HireDate)
Lecturer(EmployeeID [PK,FK→Staff], TeachingLoad, OfficeRoom)
```

PART B — AGGREGATION: Research Grant Oversight

The university has three entities: ResearchProject, Faculty (a sub-entity of Staff), and ExternalFunder. A Faculty member leads a ResearchProject funded by an ExternalFunder — this is the FUNDS relationship (with attributes: GrantAmount, StartDate, EndDate).

Now the university's Research Office needs to AUDIT each active funding arrangement. The Audit is a relationship between the ResearchOffice and the specific FUNDS relationship — not just between ResearchOffice and Project, or ResearchOffice and Funder. This requires AGGREGATION:

```
Aggregated entity: [Faculty] --FUNDS--> [ResearchProject] <-- funded by
[ExternalFunder]
   Attributes on FUNDS: GrantAmount, StartDate, EndDate

ResearchOffice ---AUDITS---> [FUNDS aggregate]
   Attributes on AUDITS: AuditDate, AuditorID, FindingsReport
```

Why not a ternary? A ternary relationship between ResearchOffice, ResearchProject, and ExternalFunder would lose the essential semantics: it is a SPECIFIC FUNDING ARRANGEMENT (not just any combination of the three) that is audited. The FUNDS relationship has its own identity (its own GrantAmount and date range), and AUDITS targets that specific arrangement.

Database tables for the aggregation:

```
FundingArrangement(FundingID [PK], FacultyID [FK], ProjectID [FK],
                   FunderID [FK], GrantAmount, StartDate, EndDate)

Audit(AuditID [PK], FundingID [FK], AuditDate, AuditorID [FK], FindingsReport)
```

This maps cleanly: FundingArrangement represents the aggregated FUNDS relationship as an entity; Audit references it via FundingID.

PART 3: Advanced SQL

This section covers advanced SQL features beyond basic SELECT/INSERT/UPDATE/DELETE. All examples are written in T-SQL (Microsoft SQL Server) syntax, as used in the course materials.

3.1 Creating and Managing Indexes

3.1.1 What Is an Index?

An index is a database object that speeds up data retrieval operations on a table, at the cost of additional storage and slightly slower write operations (INSERT/UPDATE/DELETE must update the index). An index works similarly to the index in a book — instead of scanning every page, you jump directly to the relevant entry.

3.1.2 Types of Indexes

Index Type	Description	When to Use
Clustered Index	Sorts and stores the data rows in the table based on the key. One per table. Automatically created with PRIMARY KEY.	On the column most frequently used for range queries or sorting
Non-Clustered Index	Creates a separate structure from data rows; contains pointers back to the actual data rows. Up to 999 per table in SQL Server.	On columns used frequently in WHERE, JOIN, or ORDER BY clauses
Unique Index	Ensures all values in the indexed column(s) are unique. Created automatically with UNIQUE constraint.	On columns that must hold unique values (e.g., Email, SSN)
Composite Index	An index on two or more columns. Column order matters.	When queries frequently filter on multiple columns together
Full-Text Index	Optimised for text searching within large character columns.	For CONTAINS or FREETEXT queries on VARCHAR/NVARCHAR columns
Filtered Index	Index on a subset of rows based on a WHERE predicate.	When queries target a specific subset of data (e.g., active records)

3.1.3 Creating Indexes – Syntax

```
-- Basic non-clustered index
CREATE INDEX idx_employee_lastname
ON Employee (LastName);
```

```
-- Unique index
CREATE UNIQUE INDEX idx_employee_email
ON Employee (Email);
```

```sql
-- Composite index (column order matters: LastName filtered first)
CREATE INDEX idx_emp_name
ON Employee (LastName, FirstName);
```

```sql
-- Filtered index (only index active employees)
CREATE INDEX idx_active_employees
ON Employee (DepartmentID)
WHERE Status = 'Active';
```

```sql
-- Drop an index
DROP INDEX idx_employee_lastname ON Employee;
```

```sql
-- View all indexes on a table
SELECT * FROM sys.indexes WHERE object_id = OBJECT_ID('Employee');
```

3.1.4 Index Best Practices

- Index columns that appear frequently in WHERE, JOIN ON, and ORDER BY clauses.
- Avoid indexing columns with very low cardinality (e.g., a 'Gender' column with only M/F) — the optimizer may not use the index.
- Avoid over-indexing: every index slows down INSERT, UPDATE, and DELETE.
- Rebuild or reorganise fragmented indexes periodically: ALTER INDEX idx_name ON table REBUILD;

3.2 COUNT Function

3.2.1 Overview

COUNT is an aggregate function that counts rows. It is one of the most frequently used SQL functions.

Variant	Syntax	Behaviour
COUNT(*)	COUNT(*)	Counts ALL rows, including those with NULLs
COUNT(column)	COUNT(ColumnName)	Counts only rows where the column is NOT NULL
COUNT(DISTINCT column)	COUNT(DISTINCT ColumnName)	Counts unique non-NULL values in the column

3.2.2 COUNT Examples

```sql
-- Count all rows in the Customer table
SELECT COUNT(*) AS TotalCustomers FROM Customer;
```

```sql
-- Count customers who have a phone number (ignores NULLs)
```

```sql
SELECT COUNT(PhoneNumber) AS CustomersWithPhone FROM Customer;
```

```sql
-- Count distinct countries
SELECT COUNT(DISTINCT Country) AS UniqueCountries FROM Customer;
```

```sql
-- Count orders per customer
SELECT CustomerID, COUNT(*) AS TotalOrders
FROM Orders
GROUP BY CustomerID;
```

```sql
-- Conditional count using CASE
SELECT
  COUNT(CASE WHEN Salary > 50000 THEN 1 END) AS HighEarners,
  COUNT(CASE WHEN Salary <= 50000 THEN 1 END) AS LowEarners
FROM Employee;
```

3.3 ORDER BY Clause

3.3.1 Purpose and Syntax

ORDER BY sorts the result set of a query. Without ORDER BY, SQL provides no guarantee on the order of rows. It is always the last clause in a SELECT statement.

```sql
SELECT column1, column2
FROM TableName
[WHERE condition]
[GROUP BY ...]
[HAVING ...]
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

3.3.2 ORDER BY Examples

```sql
-- Sort employees by last name ascending (default)
SELECT EmployeeID, FirstName, LastName FROM Employee
ORDER BY LastName;
```

```sql
-- Sort by salary descending
SELECT EmployeeID, Salary FROM Employee
ORDER BY Salary DESC;
```

```sql
-- Multi-column sort: dept ascending, then salary descending within dept
SELECT DeptID, EmployeeID, Salary FROM Employee
ORDER BY DeptID ASC, Salary DESC;
```

```sql
-- Sort by column position (column 2 = LastName in SELECT list)
SELECT EmployeeID, LastName, FirstName FROM Employee
ORDER BY 2 ASC, 3 ASC;
```

```sql
-- ORDER BY with TOP (get top 5 highest-paid)
SELECT TOP 5 EmployeeID, LastName, Salary FROM Employee
ORDER BY Salary DESC;
```

```sql
-- ORDER BY with OFFSET-FETCH (pagination: page 2, 10 rows per page)
SELECT EmployeeID, LastName, Salary FROM Employee
ORDER BY Salary DESC
OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY;
```

3.3.3 Key Rules for ORDER BY

- ORDER BY can reference columns not in the SELECT list (exception: when using DISTINCT, all ORDER BY columns must be in SELECT).
- ORDER BY is the only reliable way to guarantee row order in SQL.
- NULL values sort first in ASC order (treated as lowest), last in DESC in most DBMS.
- Sorting by column position (e.g., ORDER BY 2) is valid but fragile — column positions change if SELECT is modified.

3.4 HAVING Clause

3.4.1 HAVING vs. WHERE

The HAVING clause filters groups after GROUP BY has been applied, whereas WHERE filters individual rows before grouping. A key rule: aggregate functions (COUNT, SUM, AVG, MAX, MIN) cannot be used in WHERE — they must be used in HAVING.

Clause	Filters	Aggregate Functions Allowed?	Execution Order
WHERE	Individual rows BEFORE grouping	No	Step 2 (after FROM)
HAVING	Groups AFTER GROUP BY	Yes	Step 5 (after GROUP BY)

3.4.2 HAVING Examples

```sql
-- Departments with more than 5 employees
SELECT DepartmentID, COUNT(*) AS EmpCount
FROM Employee
GROUP BY DepartmentID
HAVING COUNT(*) > 5;
```

```sql
-- Job titles with average salary above 45,000 (ordered by avg salary)
SELECT JobTitle, AVG(Salary) AS AvgSalary
FROM EmployeeDemographics ED
JOIN EmployeeSalary ES ON ED.EmployeeID = ES.EmployeeID
```

```
GROUP BY JobTitle
HAVING AVG(Salary) > 45000
ORDER BY AVG(Salary) DESC;
```

```
-- Using both WHERE and HAVING together
-- WHERE filters rows first (active only), HAVING filters groups
SELECT DepartmentID, COUNT(*) AS ActiveCount, SUM(Salary) AS TotalSalary
FROM Employee
WHERE Status = 'Active'
GROUP BY DepartmentID
HAVING SUM(Salary) > 200000;
```

```
-- HAVING with multiple conditions
SELECT JobTitle, COUNT(*) AS Headcount, AVG(Salary) AS AvgSal
FROM EmployeeSalary
GROUP BY JobTitle
HAVING COUNT(*) > 1 AND AVG(Salary) > 40000;
```

3.5 DROP Statement & Variants

3.5.1 DROP vs. TRUNCATE vs. DELETE

Command	Removes	Rollback?	Resets Identity?	Triggers Fired?
DELETE	Rows (with or without WHERE)	Yes (DML – transactional)	No	Yes
TRUNCATE	All rows in the table	Yes (in SQL Server)	Yes	No
DROP TABLE	Entire table structure + data	No (DDL)	N/A	No
DROP DATABASE	Entire database	No (DDL)	N/A	No
DROP INDEX	Index object only (data intact)	No	N/A	No
DROP VIEW	View definition (base data intact)	No	N/A	No

3.5.2 DROP Syntax Examples

```
-- Drop a table (irreversible)
DROP TABLE Employee;
```

```
-- Safe drop: only drop if the table exists
DROP TABLE IF EXISTS #TempEmployee;
```

```
-- Drop a database
```

```
DROP DATABASE SalesDB;
```

```
-- Drop an index
DROP INDEX idx_employee_lastname ON Employee;
```

```
-- Drop a view
DROP VIEW vw_CustomerSales;
```

```
-- Drop a stored procedure
DROP PROCEDURE IF EXISTS usp_GetEmployeeDetails;
```

```
-- Drop a column (ALTER TABLE with DROP COLUMN)
ALTER TABLE Employee DROP COLUMN MiddleName;
```

3.5.3 DROP with Foreign Key Dependencies

You cannot drop a table that is referenced by a foreign key in another table. You must first either drop the referencing table or remove the foreign key constraint:

```
-- Remove FK constraint first
ALTER TABLE Orders DROP CONSTRAINT FK_Orders_Customer;
-- Then drop the referenced table
DROP TABLE Customer;
```

3.6 CHECK Constraint

3.6.1 Purpose

The CHECK constraint enforces a condition on the values that can be inserted or updated in a column. It is a DDL-level data validation mechanism that runs at the database engine layer — meaning it applies regardless of the application inserting the data.

3.6.2 CHECK Constraint Syntax

```
-- CHECK at column level (inline)
CREATE TABLE Employee (
    EmployeeID    INT            PRIMARY KEY,
    Age           INT            CHECK (Age >= 18 AND Age <= 65),
    Salary        DECIMAL(10,2)  CHECK (Salary > 0),
    Gender        CHAR(1)        CHECK (Gender IN ('M', 'F', 'O')),
    HireDate      DATE,
    TerminationDate DATE,
    CONSTRAINT chk_dates CHECK (TerminationDate IS NULL OR TerminationDate >
HireDate)
);
```

```
-- Adding CHECK constraint to existing table
ALTER TABLE Employee
```

```
ADD CONSTRAINT chk_salary CHECK (Salary BETWEEN 1000 AND 500000);
```

```
-- Dropping a CHECK constraint
ALTER TABLE Employee DROP CONSTRAINT chk_salary;
```

```
-- Disable a constraint temporarily (for bulk loads)
ALTER TABLE Employee NOCHECK CONSTRAINT chk_salary;
-- Re-enable it
ALTER TABLE Employee CHECK CONSTRAINT chk_salary;
```

3.6.3 Complex CHECK Constraints

```
-- Ensure discount is valid only if quantity is large enough
CREATE TABLE OrderLine (
    OrderID    INT,
    ProductID  INT,
    Quantity   INT  CHECK (Quantity > 0),
    Discount   DECIMAL(5,2) CHECK (Discount >= 0 AND Discount <= 0.50),
    CONSTRAINT chk_discount_qty
        CHECK (Discount = 0 OR Quantity >= 10)
);
```

3.6.4 CHECK vs. Other Constraints

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row; no NULLs allowed	EmployeeID INT PRIMARY KEY
UNIQUE	Ensures all values in column are distinct; NULLs allowed (one)	Email VARCHAR(100) UNIQUE
NOT NULL	Prevents NULL values in a column	LastName VARCHAR(50) NOT NULL
FOREIGN KEY	Enforces referential integrity between tables	DeptID INT REFERENCES Department(DeptID)
DEFAULT	Provides a default value if none is supplied	Country VARCHAR(50) DEFAULT 'Malaysia'
CHECK	Validates data against a logical condition	Age INT CHECK (Age >= 0)

3.7 SQL Execution Order (Logical)

Understanding the logical order SQL processes a query is critical for writing correct queries:

Step	Clause	What Happens
1	FROM / JOIN	Identifies source tables and joins them
2	WHERE	Filters rows based on row-level conditions

3	GROUP BY	Groups remaining rows by specified columns
4	HAVING	Filters groups based on aggregate conditions
5	SELECT	Computes expressions and selects columns
6	DISTINCT	Removes duplicate rows from result
7	ORDER BY	Sorts the final result set
8	TOP / OFFSET-FETCH	Limits the number of rows returned

🌐 REAL-WORLD EXAMPLE: Shopee / Lazada-Style E-Commerce Platform — Advanced SQL in Production

Scenario: An e-commerce platform has millions of products, customers, and orders. The following demonstrates real production use of INDEX, COUNT, ORDER BY, HAVING, DROP, and CHECK in this high-traffic environment.

DATABASE SCHEMA (simplified):

```
Product(ProductID, CategoryID, SellerID, Name, Price, StockQty, IsActive)
Customer(CustomerID, Name, Email, City, RegisteredDate)
Orders(OrderID, CustomerID, OrderDate, TotalAmount, Status)
OrderLine(OrderID, ProductID, Quantity, UnitPrice)
Review(ReviewID, ProductID, CustomerID, Rating, ReviewDate, ReviewText)
```

1. INDEXES — Optimising the Product Search Page
The most common query is 'show all active products in a category, sorted by price'. Without indexes, this requires a full table scan of millions of rows:

```
-- Without index: full scan of Products table (millions of rows) -- SLOW
SELECT ProductID, Name, Price FROM Product
WHERE CategoryID = 42 AND IsActive = 1
ORDER BY Price ASC;

-- Solution: Composite filtered index
CREATE INDEX idx_product_cat_price
ON Product (CategoryID ASC, Price ASC)
WHERE IsActive = 1;  -- filtered: only index active products

-- Second common pattern: seller looking up their own products
CREATE INDEX idx_product_seller
ON Product (SellerID, IsActive)
INCLUDE (Name, Price, StockQty);  -- covering index: no table lookup needed
```

2. COUNT — Daily Seller Dashboard Report

```
-- How many orders did each seller receive today?
-- How many distinct customers bought from them?
SELECT
  p.SellerID,
  COUNT(DISTINCT o.OrderID)     AS TotalOrders,
  COUNT(DISTINCT o.CustomerID)  AS UniqueCustomers,
  SUM(ol.Quantity)              AS UnitsSold,
```

```
  SUM(ol.Quantity * ol.UnitPrice) AS Revenue
FROM Orders o
JOIN OrderLine ol ON o.OrderID = ol.OrderID
JOIN Product   p  ON ol.ProductID = p.ProductID
WHERE CAST(o.OrderDate AS DATE) = CAST(GETDATE() AS DATE)
GROUP BY p.SellerID
ORDER BY Revenue DESC;
```

3. ORDER BY with OFFSET-FETCH — Product Listing Pagination

```
-- Page 3 of search results (10 items per page, sorted by popularity)
SELECT ProductID, Name, Price,
       COUNT(ol.OrderID) AS TimesSold
FROM Product p
LEFT JOIN OrderLine ol ON p.ProductID = ol.ProductID
WHERE p.CategoryID = 42 AND p.IsActive = 1
GROUP BY p.ProductID, p.Name, p.Price
ORDER BY TimesSold DESC, p.Price ASC
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;  -- page 3 (rows 21-30)
```

4. HAVING — Find Products Worth Featuring (high rating, enough reviews)

```
-- Products with average rating >= 4.5 AND at least 50 reviews
-- WHERE cannot be used here because AVG and COUNT are aggregate functions
SELECT r.ProductID, p.Name,
       COUNT(r.ReviewID)   AS ReviewCount,
       AVG(r.Rating * 1.0) AS AvgRating
FROM Review r
JOIN Product p ON r.ProductID = p.ProductID
WHERE p.IsActive = 1               -- row-level filter (before grouping)
GROUP BY r.ProductID, p.Name
HAVING COUNT(r.ReviewID) >= 50     -- group-level filter (after grouping)
   AND AVG(r.Rating * 1.0) >= 4.5
ORDER BY AvgRating DESC, ReviewCount DESC;
```

5. CHECK Constraints — Data Integrity at Database Level

```
-- Prevent bad data regardless of application bugs
CREATE TABLE Product (
  ProductID   INT           PRIMARY KEY,
  Price       DECIMAL(10,2) NOT NULL CHECK (Price > 0),
  StockQty    INT           NOT NULL CHECK (StockQty >= 0),
  Rating      DECIMAL(3,1)  CHECK (Rating BETWEEN 0.0 AND 5.0),
  Discount    DECIMAL(5,2)  DEFAULT 0 CHECK (Discount >= 0 AND Discount < 1.0),
  CONSTRAINT chk_discounted_price
    CHECK (Discount = 0 OR Price >= 5.00)  -- no discount on items < RM5
);
```

6. DROP — End-of-Year Data Archiving

```
-- Archive old orders to a history table, then drop the staging table
-- Step 1: Move 2022 orders to archive
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE YEAR(OrderDate) = 2022;

-- Step 2: Delete from main table
DELETE FROM Orders WHERE YEAR(OrderDate) = 2022;
```

```sql
-- Step 3: Drop the temporary staging table used for the migration
DROP TABLE IF EXISTS Orders_Migration_Staging;

-- Step 4: Drop an old unused index (no longer needed after schema change)
DROP INDEX idx_old_product_name ON Product;
```

PART 4: Normalisation and Denormalisation

4.1 What Is Normalisation?

Normalisation is the process of organising a relational database schema to reduce data redundancy and improve data integrity. It was developed by Edgar F. Codd (the inventor of the relational model) and involves decomposing tables into smaller, well-structured tables while preserving all data and relationships.

Normalisation is guided by Normal Forms (NF) — a set of rules or criteria. A table is said to be in a particular normal form if it satisfies all the requirements up to that level.

Anomalies That Normalisation Prevents

- Insertion Anomaly: Cannot insert data without inserting other unrelated data. E.g., cannot add a new department unless at least one employee is assigned.
- Update Anomaly: Updating one piece of data requires multiple row updates, risking inconsistency. E.g., changing a department name requires updating every employee row.
- Deletion Anomaly: Deleting a row removes unintended data. E.g., deleting the last employee in a department also loses all department information.

4.2 First Normal Form (1NF)

Rules for 1NF

14. Every column must contain atomic (indivisible) values — no multi-valued attributes.
15. Every column must contain values of a single type.
16. Each column must have a unique name.
17. Order of rows and columns does not matter.
18. The table must have a primary key.

1NF Example

BEFORE (violates 1NF — PhoneNumbers is multi-valued):

StudentID	Name	PhoneNumbers
S001	Alice	0112233, 0224455
S002	Bob	0335566

AFTER (1NF — split into separate rows):

StudentID	Name	PhoneNumber
S001	Alice	0112233
S001	Alice	0224455
S002	Bob	0335566

Better approach: Create a separate Student_Phone table with (StudentID, PhoneNumber) as composite PK.

4.3 Second Normal Form (2NF)

Rules for 2NF

A table is in 2NF if it is in 1NF AND every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). Partial dependency occurs only when the PK is composite.

2NF Example

BEFORE (violates 2NF — StudentName depends only on StudentID, not on the full PK):

StudentID (PK)	CourseID (PK)	StudentName	CourseName	Grade
S001	C01	Alice	Database Systems	A
S001	C02	Alice	Algorithms	B
S002	C01	Bob	Database Systems	C

AFTER (2NF — decomposed into three tables):

```
Student(StudentID [PK], StudentName)
Course(CourseID [PK], CourseName)
Enrollment(StudentID [PK,FK], CourseID [PK,FK], Grade)
```

4.4 Third Normal Form (3NF)

Rules for 3NF

A table is in 3NF if it is in 2NF AND no non-key attribute is transitively dependent on the primary key. A transitive dependency means: A → B → C, where A is the PK but B (a non-key attribute) determines C (another non-key attribute).

3NF Example

BEFORE (violates 3NF — ZipCode → City is a transitive dependency):

EmployeeID (PK)	Name	ZipCode	City
E001	Alice	50450	Kuala Lumpur
E002	Bob	50450	Kuala Lumpur
E003	Carol	10050	Georgetown

AFTER (3NF — transitive dependency removed):

```
Employee(EmployeeID [PK], Name, ZipCode [FK])
```

```
ZipCode_City(ZipCode [PK], City)
```

4.5 Boyce-Codd Normal Form (BCNF)

BCNF (also called 3.5NF) is a stronger version of 3NF. A table is in BCNF if for every non-trivial functional dependency X → Y, X must be a superkey. BCNF differs from 3NF when there are multiple overlapping candidate keys.

Example: A table Teaching(Student, Subject, Teacher) where a teacher teaches only one subject but a student can have multiple teachers. If Teacher → Subject, then Teacher is not a superkey — this violates BCNF but may satisfy 3NF.

Normal Form	Condition Met	What It Eliminates
1NF	Atomic values, single-type columns, PK exists	Repeating groups, multi-valued attributes
2NF	1NF + no partial dependencies	Dependencies on part of a composite key
3NF	2NF + no transitive dependencies	Dependencies between non-key attributes
BCNF	Every determinant is a superkey	Anomalies that 3NF misses with overlapping candidate keys

4.6 Denormalisation

4.6.1 What Is Denormalisation?

Denormalisation is the intentional introduction of redundancy into a database design to improve read performance. It is applied after normalisation, typically when query performance is identified as a bottleneck. Denormalisation trades storage space and write complexity for faster reads.

4.6.2 Common Denormalisation Techniques

Technique	Description	Benefit	Cost
Storing derived data	Pre-compute and store aggregate values (e.g., TotalSales in Customer table)	Avoid expensive recalculation	Must update on every change
Combining tables	Merge two frequently joined tables into one	Eliminate JOIN overhead	Redundant data, update anomalies
Adding redundant columns	Duplicate a column from a related table (e.g., store DeptName in Employee)	Single-table reads	Data inconsistency risk
Pre-joining tables	Store results of frequent joins as a materialised view or table	Fast reads for reporting	Storage cost; refresh needed

Splitting tables (horizontal)	Partition a large table into smaller ones by row range or date	Better partition elimination	More complex queries

4.6.3 Normalisation vs Denormalisation Decision

Factor	Favour Normalisation	Favour Denormalisation
Workload type	OLTP (many small transactions)	OLAP/Reporting (large reads)
Data consistency	Critical requirement	Acceptable with controls
Storage	Limited storage	Abundant storage
Write frequency	High (frequent updates/inserts)	Low (read-heavy)
Query complexity	Acceptable JOINs	JOINs are a bottleneck

🌐 REAL-WORLD EXAMPLE: AirAsia-Style Airline Booking System — Normalisation Step by Step

Scenario: An airline starts with a single flat table exported from a legacy spreadsheet system. We will normalise it all the way to 3NF, identifying every anomaly at each step.

STEP 0 — THE ORIGINAL FLAT TABLE (completely unnormalised):

BookingRef	FlightNo	FlightDate	DepartCity	ArriveCity	AircraftType	PilotName	PassengerIDs	PassengerNames	SeatNos	FareClass	TicketPrice	AgencyID
BK001	AK123	2025-03-01	KUL	SIN	A320	Capt. Ali	P01,P02	Alice, Bob	12A, 12B	Economy	199.00	AG01
BK002	AK123	2025-03-01	KUL	SIN	A320	Capt. Ali	P03	Carol	14C	Business	599.00	AG02
BK003	AK456	2025-03-02	KUL	BKK	B737	Capt. Mei	P01	Alice	22F	Economy	249.00	AG01

Problems identified: (1) PassengerIDs and PassengerNames are multi-valued (violates 1NF). (2) AgencyName and AgencyCity depend only on AgencyID, not on the full key — partial dependency (violates 2NF). (3) DepartCity, ArriveCity, AircraftType, PilotName depend only on FlightNo — partial dependency (violates 2NF). (4) AgencyCity depends on AgencyID which depends on BookingRef — transitive dependency (violates 3NF).

STEP 1 — CONVERT TO 1NF (make all values atomic):
Separate the multi-valued passenger data into individual rows, with a composite key:

BookingRef (PK)	PassengerID (PK)	FlightNo	FlightDate	SeatNo	FareClass	TicketPrice	AgencyID	AgencyName	AgencyCity	DepartCity	ArriveCity	AircraftTy

BK001	P01	AK123	2025-03-01	12A	Economy	199.00	AG01	Holidayz	KL	KUL	SIN	A32
BK001	P02	AK123	2025-03-01	12B	Economy	199.00	AG01	Holidayz	KL	KUL	SIN	A32
BK002	P03	AK123	2025-03-01	14C	Business	599.00	AG02	Travel Pro	JB	KUL	SIN	A32

1NF achieved: every cell is atomic. PK is (BookingRef, PassengerID). But partial and transitive dependencies remain.

STEP 2 — CONVERT TO 2NF (remove partial dependencies):
PK = (BookingRef, PassengerID). Identify what depends on only PART of the key:

```
FlightNo, FlightDate, FareClass, TicketPrice, AgencyID → depend only on BookingRef
DepartCity, ArriveCity, AircraftType, PilotName → depend only on FlightNo (not
even BookingRef)
SeatNo → depends on full (BookingRef, PassengerID) — OK, stays
```

Decompose into three tables:

```
BookingPassenger(BookingRef [PK,FK], PassengerID [PK,FK], SeatNo)

Booking(BookingRef [PK], FlightNo [FK], FlightDate, FareClass,
        TicketPrice, AgencyID [FK])

Flight(FlightNo [PK], DepartCity, ArriveCity, AircraftType, PilotName)

Passenger(PassengerID [PK], PassengerName)  -- assumed from original data
```

2NF achieved. But transitive dependencies remain in Booking (AgencyName, AgencyCity via AgencyID) and Flight (PilotName may depend on a Pilot entity).

STEP 3 — CONVERT TO 3NF (remove transitive dependencies):

```
In Booking: BookingRef → AgencyID → AgencyName, AgencyCity  (transitive)
In Flight:  FlightNo  → PilotID  → PilotName, Licence      (transitive)
```

Final 3NF schema:

```
Agency(AgencyID [PK], AgencyName, AgencyCity)

Booking(BookingRef [PK], FlightNo [FK], FlightDate, FareClass,
        TicketPrice, AgencyID [FK→Agency])

Pilot(PilotID [PK], PilotName, LicenceNumber)

Flight(FlightNo [PK], DepartCity, ArriveCity, AircraftType,
       PilotID [FK→Pilot])

Passenger(PassengerID [PK], PassengerName, PassportNo, Nationality)

BookingPassenger(BookingRef [PK,FK], PassengerID [PK,FK], SeatNo)
```

Result: 6 clean tables. No update anomaly (changing an agency's city requires updating one row in Agency, not hundreds of booking rows). No insertion anomaly (can add an Agency before any bookings exist). No deletion anomaly (deleting a booking does not delete flight or agency data).

DENORMALISATION DECISION: The airline's reporting dashboard needs to show 'total revenue by route per month'. This JOIN across 4 tables runs 500,000 times per day. The team creates a pre-aggregated summary table (denormalised):

```
RouteRevenueSummary(RouteKey [PK], DepartCity, ArriveCity, Month, Year,
                    TotalBookings, TotalRevenue, AvgFare)
-- Refreshed nightly by a stored procedure
-- Dashboard reads directly from this table — zero JOINs
```

PART 5: Transaction Management

5.1 What Is a Transaction?

A transaction is a logical unit of work that consists of one or more SQL statements that are executed as a single unit. Either ALL statements in the transaction succeed (commit) or ALL are undone (rollback). Transactions ensure data integrity in multi-user, concurrent database environments.

Real-world example: A bank transfer — debit Account A AND credit Account B must both succeed. If the credit fails, the debit must also be reversed.

5.2 ACID Properties

All transactions must satisfy the ACID properties to guarantee database integrity:

Property	Meaning	How Enforced in SQL Server
Atomicity	A transaction is all-or-nothing. If any part fails, the entire transaction is rolled back.	ROLLBACK statement; automatic rollback on error
Consistency	A transaction brings the database from one valid state to another; all integrity constraints must hold.	Constraints (PK, FK, CHECK, NOT NULL) and triggers
Isolation	Concurrent transactions execute as if they were sequential; changes are not visible to other transactions until committed.	Transaction isolation levels (READ COMMITTED, SERIALIZABLE, etc.)
Durability	Once a transaction is committed, its changes persist even in the event of a system crash.	Write-Ahead Logging (WAL) and transaction logs

5.3 Transaction Control in SQL

5.3.1 Basic Transaction Syntax

```
BEGIN TRANSACTION;  -- or BEGIN TRAN

UPDATE Account SET Balance = Balance - 1000 WHERE AccountID = 101;
UPDATE Account SET Balance = Balance + 1000 WHERE AccountID = 202;

-- Check for errors
IF @@ERROR <> 0
BEGIN
    ROLLBACK TRANSACTION;
    PRINT 'Transaction failed. Rolled back.';
```

```
END
ELSE
BEGIN
    COMMIT TRANSACTION;
    PRINT 'Transaction committed successfully.';
END
```

5.3.2 TRY...CATCH with Transactions (Best Practice)

```
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Account SET Balance = Balance - 500 WHERE AccountID = 101;
    UPDATE Account SET Balance = Balance + 500 WHERE AccountID = 202;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
```

5.3.3 SAVEPOINT

A savepoint creates a partial rollback point within a transaction. You can roll back to a savepoint without aborting the entire transaction:

```
BEGIN TRANSACTION;

INSERT INTO Order (OrderID, CustomerID) VALUES (501, 1001);
SAVE TRANSACTION sp_order_inserted;   -- savepoint

INSERT INTO OrderLine (OrderID, ProductID, Qty) VALUES (501, 'P01', 5);

-- If only the line insert fails:
ROLLBACK TRANSACTION sp_order_inserted;  -- rolls back only to savepoint

COMMIT TRANSACTION;  -- commits the original Order insert
```

5.4 Concurrency Problems

When multiple transactions execute simultaneously without proper control, the following problems can arise:

Problem	Description	Example
Dirty Read	A transaction reads data that has been written by another uncommitted transaction.	T1 updates Salary to 60000 but hasn't committed. T2 reads 60000. T1 rolls back — T2 read dirty data.

Non-Repeatable Read	A transaction reads the same row twice and gets different values because another transaction updated it in between.	T1 reads Balance=5000. T2 updates Balance to 4000. T1 reads Balance again: 4000.
Phantom Read	A transaction re-executes a query and finds rows that didn't exist before (inserted by another transaction).	T1 queries 'SELECT * WHERE Salary > 50000' → 5 rows. T2 inserts a new row. T1 re-queries → 6 rows.
Lost Update	Two transactions read and update the same row; one update overwrites the other.	T1 and T2 both read Balance=5000. T1 sets 5000-200=4800. T2 sets 5000-300=4700. T2's write overwrites T1's — 200 lost.
Unrepeatable Read	Similar to non-repeatable; a row read changes between two reads in the same transaction.	Covered under Non-Repeatable Read above.

5.5 Isolation Levels

SQL provides isolation levels to balance data consistency against concurrency performance:

Isolation Level	Dirty Read	Non-Repeatable	Phantom Read	Description
READ UNCOMMITTED	Possible	Possible	Possible	Lowest isolation. Reads without locks. Fastest but most risky.
READ COMMITTED	Prevented	Possible	Possible	Default in SQL Server. Reads only committed data.
REPEATABLE READ	Prevented	Prevented	Possible	Holds read locks until transaction ends.
SERIALIZABLE	Prevented	Prevented	Prevented	Highest isolation. Full serialisation. Slowest.
SNAPSHOT	Prevented	Prevented	Prevented	Uses row versioning (tempdb). Non-blocking reads.

```
-- Set isolation level
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

5.6 Locking

Lock Types

Lock Type	Symbol	Description
Shared (S)	S	For READ operations. Multiple transactions can hold shared locks simultaneously.
Exclusive (X)	X	For WRITE operations (INSERT/UPDATE/DELETE). No other lock can be held simultaneously.
Update (U)	U	Prevents deadlocks during update operations. Converts to X lock when update occurs.
Intent (I)	IS, IX, SIX	Signals intent to acquire finer-grained locks on sub-resources (page, row).
Schema (Sch)	Sch-M, Sch-S	Schema modification/stability. Held during DDL operations.

5.6.1 Deadlocks

A deadlock occurs when two or more transactions are waiting for each other to release locks, creating a circular dependency. SQL Server's lock monitor (the deadlock detector) automatically detects deadlocks and kills the transaction with the lowest cost (the 'deadlock victim'), which then receives error 1205.

Prevention strategies:

- Always access tables in the same order across all transactions.
- Keep transactions short — hold locks for the minimum time necessary.
- Use lower isolation levels where consistency requirements allow.
- Use DEADLOCK_PRIORITY to control which transaction becomes the victim.

🌐 REAL-WORLD EXAMPLE: Maybank Online Banking — Transaction Management in Production

Scenario: Maybank processes millions of transactions daily — fund transfers, bill payments, salary crediting, and loan repayments. Every operation must satisfy ACID. Below are four production scenarios demonstrating transaction management.

SCHEMA:

```
Account(AccountID, CustomerID, AccountType, Balance, Status, LastUpdated)
TransactionLog(TxnID, FromAccountID, ToAccountID, Amount, TxnType, TxnDate,
Status)
BillPayment(BillID, AccountID, BillerCode, Amount, DueDate, PaidDate)
LoanRepayment(RepaymentID, LoanID, AccountID, Principal, Interest, TotalAmount,
PaidDate)
```

SCENARIO 1: IBFT (Interbank Fund Transfer) — Atomicity in Action
Alice (Account A-001, Balance RM 5,000) transfers RM 1,500 to Bob (B-007). Both debit and credit must succeed together, or neither happens:

```
BEGIN TRY
```

```
  BEGIN TRANSACTION;
  -- Step 1: Debit sender (with insufficient-funds guard)
  UPDATE Account SET Balance = Balance - 1500, LastUpdated = GETDATE()
  WHERE AccountID = 'A-001' AND Balance >= 1500 AND Status = 'Active';
  IF @@ROWCOUNT = 0 THROW 60001, 'Debit failed: insufficient funds.', 1;
  -- Step 2: Credit recipient
  UPDATE Account SET Balance = Balance + 1500, LastUpdated = GETDATE()
  WHERE AccountID = 'B-007' AND Status = 'Active';
  IF @@ROWCOUNT = 0 THROW 60002, 'Credit failed: recipient inactive.', 1;
  -- Step 3: Write audit trail
  INSERT INTO TransactionLog VALUES
    (NEWID(),'A-001','B-007',1500.00,'IBFT',GETDATE(),'Success');
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
  INSERT INTO TransactionLog VALUES
    (NEWID(),'A-001','B-007',1500.00,'IBFT',GETDATE(),'Failed');
  PRINT 'Transfer failed: ' + ERROR_MESSAGE();
END CATCH;
```

ACID analysis of this transfer:

ACID Property	How It Is Satisfied
Atomicity	Both UPDATEs are inside one BEGIN TRANSACTION. If either fails, ROLLBACK undoes both — Alice is never debited without Bob being credited.
Consistency	Balance >= 1500 guard prevents overdraft. Status = Active check prevents payments to closed accounts. All business rules enforced at DB level.
Isolation	Under READ COMMITTED, if two transfers run simultaneously from A-001, the second waits for the first to commit before reading Balance — prevents Lost Update.
Durability	After COMMIT, the change is written to the transaction log on disk. A server crash 1 second post-COMMIT is fully recoverable via WAL replay on restart.

SCENARIO 2: Salary Batch Crediting — SAVEPOINT for Partial Recovery
At month-end, HR uploads 50,000 salary credits. If ONE account has an error (e.g., account closed), only that credit fails — the rest still commit:

```
BEGIN TRANSACTION;  -- outer batch transaction
DECLARE salary_cursor CURSOR FOR
  SELECT EmployeeID, AccountID, SalaryAmount FROM SalaryBatch WHERE
BatchID=202503;
OPEN salary_cursor; FETCH NEXT FROM salary_cursor INTO @EmpID, @AcctID, @Amount;
WHILE @@FETCH_STATUS = 0
BEGIN
  SAVE TRANSACTION sp_single_salary;   -- savepoint per employee
  BEGIN TRY
    UPDATE Account SET Balance=Balance+@Amount WHERE AccountID=@AcctID AND
Status='Active';
    IF @@ROWCOUNT=0 THROW 60003,'Account inactive',1;
    INSERT INTO TransactionLog
VALUES(NEWID(),NULL,@AcctID,@Amount,'SalaryCr',GETDATE(),'Success');
  END TRY
  BEGIN CATCH
```

```
    ROLLBACK TRANSACTION sp_single_salary;  -- undo THIS credit only
    INSERT INTO SalaryErrors VALUES(@EmpID, @AcctID, ERROR_MESSAGE(), GETDATE());
  END CATCH
  FETCH NEXT FROM salary_cursor INTO @EmpID, @AcctID, @Amount;
END
CLOSE salary_cursor; DEALLOCATE salary_cursor;
COMMIT TRANSACTION;  -- commit all successful credits
```

SCENARIO 3: Deadlock — Concurrent Loan Repayments
Two customers make simultaneous payments accessing the same rows in opposite order, causing a deadlock:

```
-- Session 1 (Customer X repaying Loan L-01):
BEGIN TRAN;
UPDATE Account SET Balance=Balance-800 WHERE AccountID='X-100';  -- locks X-100
-- << pause >> tries to lock LoanRepayment row for L-01

-- Session 2 (Customer Y — different account, same loan):
BEGIN TRAN;
UPDATE LoanRepayment SET PaidDate=GETDATE() WHERE LoanID='L-01'; -- locks L-01
-- << pause >> tries to lock Account row X-100 → DEADLOCK!
-- SQL Server kills Session 2 (victim → ERROR 1205 → auto ROLLBACK)
-- Session 1 completes successfully
-- Prevention: always access Account FIRST, then LoanRepayment, in all procedures
```

SCENARIO 4: Isolation Levels — ATM Balance vs. Teller Update

Isolation Level	Scenario & Result
READ UNCOMMITTED	ATM shows RM 3,200 while teller's update is mid-flight. Teller rolls back — customer saw a balance that never existed. DANGEROUS for banking.
READ COMMITTED (default)	ATM only reads committed balances. Safe for display. But if ATM checks balance then debits, a concurrent transaction could change it between those two reads (non-repeatable read risk).
REPEATABLE READ	ATM holds a shared lock between balance-check and debit. No other transaction can alter the row between reads. Used inside Maybank ATM withdrawal transactions.
SERIALIZABLE	End-of-day interest calculation: no new accounts can be inserted mid-run. Prevents phantom accounts being missed. Used for batch interest and regulatory reporting runs.

PART 6: Data Backup and Recovery

6.1 Importance of Backup and Recovery

Data backup and recovery is a fundamental database administration responsibility. Databases face threats from hardware failures, software bugs, human error, natural disasters, and cyberattacks (ransomware, SQL injection). A well-designed backup strategy ensures the database can be restored to a known consistent state within an acceptable time frame.

Two key metrics define recovery objectives:

- RPO (Recovery Point Objective): The maximum amount of data loss (measured in time) that is acceptable. E.g., RPO = 1 hour means no more than 1 hour of data can be lost.
- RTO (Recovery Time Objective): The maximum acceptable downtime before the system must be back online.

6.2 Backup Types

Backup Type	What Is Backed Up	Speed	Restore Complexity	Best For
Full Backup	The entire database (all data files and transaction log tail)	Slowest	Simplest — restore one file	Weekly baseline backups
Differential Backup	All data changed since the last FULL backup	Faster than full	Need full + latest differential	Daily backups between fulls
Transaction Log Backup	All transaction log records since last log backup	Fastest	Full + differential + all logs in sequence	Hourly or more frequent; supports point-in-time recovery
Copy-Only Backup	Full backup that does not affect the backup sequence (no reset of differential base)	Same as full	Same as full	Ad hoc backups without disrupting schedule
File/Filegroup Backup	Specific data files or filegroups	Varies	Complex; must sync with log backups	Very large databases (VLDB)
Partial Backup	Primary filegroup + specified read-write filegroups	Varies	Complex	Databases with read-only filegroups

6.3 Recovery Models in SQL Server

The recovery model controls how transaction logs are managed and what backup/restore options are available:

Recovery Model	Log Truncation	Log Backup	Point-in-Time	Best Use

		Required?	Recovery?	Case
SIMPLE	Auto-truncated at checkpoint	No	No — only to last backup	Dev/test; low data-loss tolerance not required
FULL	Only when log backup is taken	Yes	Yes — to any specific point	Production OLTP; minimal data loss required
BULK_LOGGED	At log backup; bulk ops minimally logged	Yes	Limited (not during bulk operations)	Period of large bulk loads (BULK INSERT, SELECT INTO)

```
-- Check recovery model
SELECT name, recovery_model_desc FROM sys.databases WHERE name = 'SalesDB';
```

```
-- Change recovery model
ALTER DATABASE SalesDB SET RECOVERY FULL;
```

6.4 SQL Server Backup Commands

6.4.1 Full Backup

```
-- Full database backup
BACKUP DATABASE SalesDB
TO DISK = 'D:\Backups\SalesDB_Full_20250101.bak'
WITH INIT,            -- Overwrite existing backup set
     NAME = 'SalesDB Full Backup',
     STATS = 10;      -- Report progress every 10%
```

6.4.2 Differential Backup

```
BACKUP DATABASE SalesDB
TO DISK = 'D:\Backups\SalesDB_Diff_20250102.bak'
WITH DIFFERENTIAL,
     NAME = 'SalesDB Differential Backup';
```

6.4.3 Transaction Log Backup

```
BACKUP LOG SalesDB
TO DISK = 'D:\Backups\SalesDB_Log_202501021200.bak'
```

```
WITH NAME = 'SalesDB Log Backup';
```

6.4.4 Restore Database

```
-- Step 1: Restore full backup (WITH NORECOVERY keeps DB in restoring state)
RESTORE DATABASE SalesDB
FROM DISK = 'D:\Backups\SalesDB_Full_20250101.bak'
WITH NORECOVERY,
     MOVE 'SalesDB' TO 'C:\Data\SalesDB.mdf',
     MOVE 'SalesDB_log' TO 'C:\Data\SalesDB_log.ldf';
```

```
-- Step 2: Apply differential backup
RESTORE DATABASE SalesDB
FROM DISK = 'D:\Backups\SalesDB_Diff_20250102.bak'
WITH NORECOVERY;
```

```
-- Step 3: Apply transaction log backups
RESTORE LOG SalesDB
FROM DISK = 'D:\Backups\SalesDB_Log_202501021200.bak'
WITH NORECOVERY;
```

```
-- Step 4: Bring database online (RECOVERY = final step)
RESTORE DATABASE SalesDB WITH RECOVERY;
```

6.4.5 Point-in-Time Recovery

```
-- Restore to a specific point in time (e.g., just before accidental DELETE)
RESTORE LOG SalesDB
FROM DISK = 'D:\Backups\SalesDB_Log_20250102.bak'
WITH STOPAT = '2025-01-02 13:45:00',
     RECOVERY;
```

6.5 Database Integrity Checks (DBCC)

DBCC (Database Console Commands) are maintenance commands used to check database integrity, perform maintenance, and gather statistics:

Command	Purpose	Example
DBCC CHECKDB	Checks logical and physical integrity of all objects in the database	DBCC CHECKDB ('SalesDB') WITH NO_INFOMSGS;
DBCC CHECKTABLE	Checks integrity of a single table and its indexes	DBCC CHECKTABLE ('Employee');
DBCC CHECKALLOC	Checks consistency of disk space allocation structures	DBCC CHECKALLOC ('SalesDB');
DBCC	Reduces the size of data and	DBCC SHRINKDATABASE ('SalesDB', 10);

SHRINKDATABASE	log files	
DBCC SHRINKFILE	Shrinks a specific database file	DBCC SHRINKFILE (SalesDB_log, 50);
DBCC UPDATEUSAGE	Corrects inaccuracies in sys.allocation_units	DBCC UPDATEUSAGE ('SalesDB');

6.6 Backup Strategy Best Practices

- 3-2-1 Rule: Keep 3 copies of data, on 2 different media types, with 1 offsite.
- Test restores regularly — a backup that has never been tested is not a backup.
- Automate backups using SQL Server Agent Jobs or third-party tools.
- Monitor backup job success/failure and set up alerts.
- Store backups encrypted when they contain sensitive data.
- Define and document RPO and RTO targets before designing the backup strategy.
- Consider availability groups and log shipping for high-availability scenarios.

Scenario	Recommended Strategy
Small DB, low change frequency	Weekly Full + Daily Differential. SIMPLE recovery model.
Medium OLTP, moderate changes	Daily Full + Hourly Differential + Transaction Log every 15–30 min. FULL recovery.
Large critical OLTP (banking, healthcare)	Full + Differential + Log every 5 minutes. Mirroring or Always On AG. FULL recovery.
Data Warehouse / Reporting DB	Weekly Full + Differential after major loads. BULK_LOGGED during ETL.
Development / Test DB	Weekly Full backup is usually sufficient. SIMPLE recovery.

🌐 REAL-WORLD EXAMPLE: Pantai Hospital Group — Data Backup & Disaster Recovery Plan

Scenario: Pantai Hospital Group operates 12 hospitals across Malaysia. Their SQL Server database stores patient records, prescriptions, lab results, billing, and staff rosters — 24/7 critical systems where data loss or prolonged downtime is life-threatening and legally non-compliant. Here is their complete, realistic backup and recovery plan.

STEP 1 — DEFINE RECOVERY OBJECTIVES

Database	System Type	RPO (Max Data Loss)	RTO (Max Downtime)	Justification
PatientRecords	Critical OLTP	5 minutes	15 minutes	Patient safety — missing prescription or allergy data can be fatal
Billing	OLTP	30 minutes	1 hour	Financial impact;

				delays acceptable but not data loss
LabResults	OLTP	15 minutes	30 minutes	Results affect treatment decisions — must be near-current
HR_Payroll	OLTP	1 hour	4 hours	Monthly payroll cycle; short delays tolerable
Analytics_DW	Data Warehouse	24 hours	8 hours	Reporting; one day old data acceptable

STEP 2 — CONFIGURE RECOVERY MODELS

```
-- Critical databases use FULL recovery model
ALTER DATABASE PatientRecords SET RECOVERY FULL;
ALTER DATABASE Billing        SET RECOVERY FULL;
ALTER DATABASE LabResults     SET RECOVERY FULL;
ALTER DATABASE HR_Payroll     SET RECOVERY FULL;

-- Analytics warehouse uses BULK_LOGGED during ETL loads, then switches back
ALTER DATABASE Analytics_DW   SET RECOVERY BULK_LOGGED;  -- during ETL window
ALTER DATABASE Analytics_DW   SET RECOVERY FULL;          -- after ETL completes
```

STEP 3 — BACKUP SCHEDULE (SQL Server Agent Jobs)

Database	Full Backup	Differential Backup	Transaction Log Backup	Retention
PatientRecords	Sunday 01:00 AM	Daily 11:00 PM	Every 5 minutes	Full: 8 weeks; Diff: 2 weeks; Log: 30 days
Billing	Sunday 02:00 AM	Daily 11:30 PM	Every 30 minutes	Full: 4 weeks; Diff: 2 weeks; Log: 14 days
LabResults	Sunday 01:30 AM	Daily 11:15 PM	Every 15 minutes	Full: 4 weeks; Diff: 2 weeks; Log: 14 days
HR_Payroll	Sunday 03:00 AM	Weekly Wednesday	Every 1 hour	Full: 12 weeks; Log: 30 days
Analytics_DW	Sunday 04:00 AM	After each ETL load	Not applicable	Full: 4 weeks

STEP 4 — BACKUP COMMANDS (PatientRecords example)

```
-- Sunday Full Backup Job
BACKUP DATABASE PatientRecords
TO DISK = '\\NAS01\SQLBackups\PatientRecords\Full\PR_Full_'
        + CONVERT(VARCHAR,GETDATE(),112) + '.bak'
WITH INIT, COMPRESSION,
     ENCRYPTION (ALGORITHM = AES_256,
                 SERVER CERTIFICATE = BackupCert),  -- HIPAA/PDPA compliance
```

```
     NAME = 'PatientRecords Full Backup',
     STATS = 5;

-- Every-5-minute Transaction Log Backup Job
BACKUP LOG PatientRecords
TO DISK = '\\NAS01\SQLBackups\PatientRecords\Log\PR_Log_'
        + CONVERT(VARCHAR,GETDATE(),112)
        + '_' + REPLACE(CONVERT(VARCHAR,GETDATE(),108),':','') + '.bak'
WITH COMPRESSION, ENCRYPTION (ALGORITHM=AES_256, SERVER CERTIFICATE=BackupCert);
```

STEP 5 — 3-2-1 STORAGE STRATEGY

Copy	Location	Media Type	Sync Frequency	Purpose
Copy 1 (Primary)	Production SQL Server local SSD	NVMe SSD	Continuous (live data)	Operational reads/writes
Copy 2 (On-site backup)	Dedicated Backup NAS (NAS01) in server room	RAID-6 NAS array	Every 5 min (log), daily (full)	Fast local restore within minutes
Copy 3 (Offsite/Cloud)	Azure Blob Storage (RA-GRS tier)	Cloud object storage	Nightly sync of all backups	Disaster recovery if server room destroyed (fire, flood, theft)

Azure immutable storage (WORM policy) is enabled — ransomware cannot delete or encrypt cloud backups even if on-premise servers are fully compromised.

STEP 6 — DISASTER RECOVERY DRILL (Simulated ransomware attack, 3:47 AM on a Tuesday)
Timeline: Attack discovered at 04:00 AM. All on-premise servers encrypted. Team initiates recovery from Azure cloud backups:

Time	Action	Duration
04:05 AM	Spin up new Azure VM with SQL Server 2022; download Sunday Full Backup from Azure Blob	20 min
04:25 AM	RESTORE DATABASE PatientRecords FROM DISK='..Sunday_Full..' WITH NORECOVERY	15 min
04:40 AM	RESTORE Monday Differential Backup WITH NORECOVERY	8 min
04:48 AM	Apply 23 Transaction Log backups (Monday 11:15 PM through Tuesday 03:45 AM) WITH NORECOVERY each	12 min
05:00 AM	RESTORE LOG PatientRecords WITH STOPAT='2025-03-18 03:47:00', RECOVERY — brings DB online	2 min
05:02 AM	DBCC CHECKDB to verify integrity	5 min
05:07 AM	Update DNS/connection strings to point to recovery VM	3 min
05:10 AM	System ONLINE — 1 hr 10 min total downtime; data loss: 2 minutes (last log was 03:45 AM)	—

STEP 7 — INTEGRITY MONITORING (Weekly DBCC CHECKDB Job)

```
-- Runs every Sunday at 00:00 AM, before the full backup
USE PatientRecords;
DBCC CHECKDB ('PatientRecords')
  WITH NO_INFOMSGS, ALL_ERRORMSGS, DATA_PURITY;

-- If errors found, send alert email and halt the backup job
IF @@ERROR <> 0
BEGIN
  EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'DBA_Alerts',
    @recipients   = 'dba-team@pantaihospital.com.my',
    @subject      = 'CRITICAL: DBCC CHECKDB Errors on PatientRecords',
    @body         = 'Integrity check failed. DO NOT rely on existing backups.';
END
```

KEY LESSONS from this real-world scenario:

- RPO of 5 minutes for PatientRecords is achieved by running transaction log backups every 5 minutes (not hourly or daily).
- RTO of 15 minutes is achievable only because restore procedures are scripted, tested monthly, and the team knows exactly what to do.
- Encryption of backups is mandatory — patient data is protected under Malaysia's PDPA and healthcare regulations even in backup files.
- Cloud immutability (WORM) saved the hospital — ransomware encrypted on-premise backups but could not touch Azure immutable blobs.
- DBCC CHECKDB runs BEFORE backups — backing up a corrupted database is worse than useless.
- The 2-minute data loss (03:45–03:47 AM) was acceptable per the 5-minute RPO agreed with hospital management.

PART 7: Examination Questions

The following 60 scenario-based examination questions (10 per topic) are designed to test deep understanding of concepts and their application. Each question is followed by a model answer.

Section A: Entity Relationship Model & Attributes (10 Questions)

Q1. A university needs to store student records. Each student has a student ID, full name (first and last), multiple email addresses, and a date of birth. The age should be displayed in queries but never stored. Design the entity with appropriate attributes and justify your choices.

Answer: StudentID: Key attribute (simple, stored). FullName: Composite attribute → split into FirstName, LastName. Email: Multi-valued attribute → handled via a separate table Student_Email(StudentID, Email). DateOfBirth: Stored attribute. Age: Derived attribute (calculated from DOB using DATEDIFF — not stored to avoid inconsistency).

Q2. In an ER diagram for an online store, should 'Address' be modelled as an attribute of Customer, or as a separate entity? Justify your answer.

Answer: Address should be a separate entity if: (1) a customer can have multiple addresses (shipping vs. billing — multi-valued), or (2) queries need to filter/search by address components (city, postal code — structure matters). If only one address is ever needed and no structured querying is required, a composite attribute suffices. For most real-world stores, Address as a separate entity is the better design.

Q3. A hospital database has a Doctor entity and a Patient entity. A Doctor can treat many Patients, and a Patient can be treated by many Doctors. What is the cardinality of this relationship, and how would you implement it in a relational schema?

Answer: This is a Many-to-Many (M:N) relationship. Implementation: Create a junction table Treats(DoctorID [FK], PatientID [FK], TreatmentDate, Diagnosis) with a composite primary key (DoctorID, PatientID, TreatmentDate) or a surrogate key. DoctorID references Doctor and PatientID references Patient.

Q4. Explain the difference between total and partial participation with a concrete example from a company database.

Answer: Total participation (double line): every entity instance must participate. Example: every Employee MUST belong to a Department — an Employee cannot exist without being assigned to one. Partial participation (single line): some instances may not participate. Example: a Department may or may not have a Manager — not all departments have a manager assigned.

Q5. A student's record contains the following data: StudentID=S01, Courses='Math, Science, History'. Does this violate any normal form, and what attribute type problem does it represent?

Answer: Yes — this violates 1NF. The Courses column contains multiple values (not atomic). This is a multi-valued attribute. Solution: Create a separate table Enrollment(StudentID [FK], CourseName) with composite PK (StudentID, CourseName), or a Course entity with its own CourseID.

Q6. An ER diagram shows an Employee entity with a weak entity called Dependent. Describe the characteristics of a weak entity and how it differs from a strong entity in both ER notation and relational schema.

Answer: Weak entity: (1) Cannot be uniquely identified without its owner (Employee). (2) Has a partial key (DependentName) represented with a dashed underline. (3) Connected to its owner via an identifying relationship (double diamond). (4) Represented with a double rectangle. In relational schema: Dependent(DependentName [partial key], EmployeeID [FK to Employee]) where PK = (DependentName, EmployeeID).

Q7. What is a composite key? Give an example and explain when you would use it over a surrogate key.

Answer: A composite key is a primary key made up of two or more columns. Example: Enrollment(StudentID, CourseID) — neither alone uniquely identifies an enrollment, but together they do. Use a composite key when the combination of attributes is naturally unique and meaningful. Use a surrogate key (auto-generated integer) when the natural composite is too wide, changes frequently, or is used heavily in foreign key references.

Q8. In a library system, a Book can have multiple Authors and an Author can write multiple Books. How would you represent this in an ER diagram and in a relational schema? What additional attributes might the relationship itself carry?

Answer: ER Diagram: Book (BookID, Title) and Author (AuthorID, Name) connected by a many-to-many relationship 'Wrote'. Relational schema: BookAuthor(BookID [PK,FK], AuthorID [PK,FK], ContributionType, ChapterWritten). The relationship itself can carry attributes like ContributionType (primary author, co-author, editor) and order of authorship.

Q9. Distinguish between a candidate key and an alternate key using the Employee table as an example. Why does this distinction matter?

Answer: A candidate key is any minimal set of attributes that can uniquely identify a row. An alternate key is any candidate key that was NOT chosen as the primary key. In Employee: EmployeeID and NationalID and Email might all be candidate keys. If EmployeeID is chosen as the primary key, then NationalID and Email become alternate keys. This matters for: (1) enforcing UNIQUE constraints on alternates, (2) documenting business rules, and (3) choosing JOIN columns.

Q10. A retail company has a Sale entity with attributes: SaleID, SaleDate, CustomerName, CustomerCity, ProductName, ProductPrice, Quantity, TotalAmount. Identify all the types of attributes present and flag any that may cause normalisation problems.

Answer: SaleID: Simple, key. SaleDate: Simple, stored. CustomerName: Simple (but should be FK to Customer). CustomerCity: Simple (transitive dependency via CustomerID — normalisation issue). ProductName: Simple (should be FK to Product). ProductPrice: Simple (stored; may change over time — consider historical pricing). Quantity: Simple, stored. TotalAmount: Derived (= ProductPrice × Quantity — should not be stored). Issues: CustomerCity creates a transitive dependency; ProductName/Price belong in a Product table; TotalAmount should be derived.

Section B: Aggregation and ISA Hierarchies (10 Questions)

Q1. Explain generalisation vs. specialisation with a real-world example from a transportation company database.

Answer: Generalisation (bottom-up): Car, Truck, and Motorcycle share attributes (VehicleID, LicensePlate, FuelType) → generalise into Vehicle superclass. Specialisation (top-down): Starting with Vehicle, specialise into Car (NumSeats), Truck (LoadCapacity), and Motorcycle (SidecarAttached). The IS-A triangle connects Vehicle to its sub-entities. Attributes of Vehicle are inherited by all sub-entities.

Q2. In an ISA hierarchy for a university Person entity, a person can simultaneously be a Student and a Staff member (e.g., a teaching assistant). What type of constraint (disjoint or overlapping) applies, and how does this affect implementation?

Answer: Overlapping constraint (o) — a person CAN belong to multiple subclasses simultaneously. Implementation: Use the Table-per-Subclass approach: Person(PersonID, Name, DOB), Student(PersonID [FK], RollNo, Major), Staff(PersonID [FK], EmpID, Department). A teaching assistant would have rows in BOTH Student and Staff tables sharing the same PersonID.

Q3. Why would aggregation be used in an ER model instead of a ternary relationship? Use the Employees-Projects-Departments example to illustrate.

Answer: Aggregation is used when we need a relationship that involves another relationship. In the example: Department SPONSORS Project (with attribute 'budget'). An Employee MONITORS this sponsorship (with attribute 'until'). Monitors cannot be connected directly to Sponsors in standard ER notation. By aggregating the Sponsors relationship into a higher-level entity, Monitors can reference it. A ternary relationship would incorrectly imply that monitoring is about Employee-Project-Department together, losing the semantics that it is specifically the sponsorship that is monitored.

Q4. A company's HR database has an Employee entity and a Manager IS-A Employee. The Manager entity has an additional attribute DiscretionaryBudget. If each manager gets one budget covering ALL departments they manage (not per department), how should this be modelled?

Answer: Use the ISA hierarchy: Create a Manager sub-entity inheriting from Employee. Store DiscretionaryBudget as an attribute of Manager, NOT as a descriptive attribute of the Manages relationship. If it were stored in the Manages relationship, it would appear once per (Manager, Department) pair — redundantly storing the same budget for each managed department. The ISA solution stores it once in the Manager sub-entity.

Q5. Explain three ways to implement an ISA hierarchy in a relational database, comparing their advantages and disadvantages.

Answer: (1) Single Table (Table-per-Hierarchy): One table with all attributes and a discriminator column (PersonType = 'Student'/'Teacher'). Pro: Simple; no JOINs. Con: Many NULL values; wastes storage. (2) Table-per-Subclass (Table-per-Type): Person table + Student table + Teacher table, linked by FK. Pro: No NULLs; clean structure. Con: Requires JOIN to get full record. (3) Table-per-Concrete-Class: Separate tables for Student and Teacher, each containing all Person attributes. Pro: No JOINs. Con: Duplicates Person attributes; hard to enforce uniqueness across subtypes.

Q6. What is attribute inheritance in the context of ISA hierarchies? Why is it important for database design?

Answer: Attribute inheritance means that all attributes and relationships of a superclass are automatically available to all its subclasses. Example: Person has Name, Age, Gender. Student (a subclass) inherits Name, Age, Gender and adds RollNo. Teacher inherits Name, Age, Gender and adds EmpID. Importance: (1) Reduces redundancy — common attributes are defined once. (2) Enforces consistency — all subclasses share the same base attributes. (3) Simplifies queries — Person-level queries apply to all subtypes.

Q7. In an ER diagram, when would you use a double line (total participation) vs. a single line (partial participation) between an entity and a relationship? Give two examples.

Answer: Total participation (double line): Every entity instance MUST participate. Example 1: Every Employee MUST work for a Department (a department-less employee is not allowed). Example 2: Every OrderLine MUST reference an Order (an order line cannot exist without an order). Partial participation (single line): Some entity instances may not participate. Example 1: A Department MAY be managed by an Employee (some departments have no manager). Example 2: A Customer MAY have placed an Order (new customers may have no orders yet).

Q8. Describe a scenario where both ISA hierarchy and aggregation would appear in the same ER diagram.

Answer: Scenario: A research university. Person IS-A hierarchy: Researcher (sub-entity) and Administrator (sub-entity). Separately, Project and Department have a Sponsors relationship (with budget). A Researcher MONITORS a sponsorship (aggregation). The same ER diagram contains: (1) ISA hierarchy for Person, and (2) Aggregation of Sponsors for the Monitors relationship. This combination models both hierarchical specialisation and complex relationship tracking.

Q9. Can a ternary relationship always be replaced by three binary relationships? Explain with an example.

Answer: No, not always. Consider: Contracts(Supplier, Department, Part, qty). The ternary means 'Department has contracted Supplier for a specific Part'. Three binaries — Supplier 'can-supply' Part, Department 'needs' Part, Department 'deals-with' Supplier — do NOT imply that Department has actually contracted Supplier for that Part. You cannot store qty in any binary. A ternary relationship cannot be replaced when all three entities jointly determine an attribute or meaning that no pair alone can capture.

Q10. A shipping company has Trucks, Drivers, and Routes. A Driver can drive many Trucks, a Truck can be used on many Routes, and a Driver drives a specific Truck on a specific Route at a specific DateTime. Model this as either aggregation or ternary and justify.

Answer: This should be modelled as a ternary relationship: Assignment(DriverID, TruckID, RouteID, DateTime). All three entities jointly participate in one assignment event. No pair alone captures the meaning: (Driver, Truck) without Route doesn't tell us where; (Truck, Route) without Driver doesn't tell us who drove. Aggregation would be appropriate if, say, the Truck-Route pairing (TruckRoute) itself had a relationship with Driver (e.g., a Driver monitors a TruckRoute over time). In this simpler case, a ternary is cleaner.

Section C: Advanced SQL (10 Questions)

Q1. A Products table has 2 million rows. Queries frequently filter by CategoryID and sort by Price DESC. Write the optimal index definition and explain why you chose it.

Answer: CREATE INDEX idx_products_cat_price ON Products (CategoryID ASC, Price DESC); — Composite index: CategoryID is the first column (equality filter in WHERE), Price is second (ORDER BY direction matches). This is a covering index for the query pattern: the database can satisfy the WHERE and ORDER BY from the index alone without accessing the base table data pages. If the SELECT also retrieves only a few columns, adding them as INCLUDE columns would make it a covering index: CREATE INDEX idx_products_cat_price ON Products (CategoryID, Price DESC) INCLUDE (ProductName, Stock);

Q2. Write a query to find all departments where the average employee salary exceeds the company-wide average salary. Use HAVING appropriately.

Answer: SELECT DepartmentID, AVG(Salary) AS DeptAvg FROM Employee GROUP BY DepartmentID HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employee) ORDER BY DeptAvg DESC; — The subquery calculates the company average. HAVING filters groups whose aggregate value exceeds it. This cannot use WHERE because AVG(Salary) is an aggregate function.

Q3. Explain the difference between DROP TABLE, TRUNCATE TABLE, and DELETE FROM table (with no WHERE clause). When would you use each?

Answer: DELETE (no WHERE): Removes all rows one-by-one, fires triggers, is fully logged, can be rolled back, does NOT reset IDENTITY. Use when you need trigger execution or rollback capability. TRUNCATE: Removes all rows in a minimally logged page-deallocation operation. Much faster than DELETE. Resets IDENTITY counter. Cannot be used if FK references the table. Can be rolled back within a transaction in SQL Server. Use for rapid bulk removal. DROP TABLE: Removes the entire table structure, all data, indexes, constraints, and triggers. Cannot be rolled back. Use only when the table itself is no longer needed.

Q4. A customer table has a CHECK constraint ensuring that customer age must be between 0 and 120. A data import job needs to load records where some ages are NULL (unknown). Will the CHECK constraint block this import? Explain.

Answer: No — CHECK constraints in SQL Server (and most RDBMS) treat NULL as UNKNOWN, and UNKNOWN is NOT considered a constraint violation. So a NULL age will pass the CHECK (Age BETWEEN 0 AND 120) constraint. The row will be inserted successfully. If you want to prevent NULLs, you must add a separate NOT NULL constraint on the Age column in addition to the CHECK constraint.

Q5. Write a query using COUNT, GROUP BY, ORDER BY, and HAVING to show job titles that have more than 2 employees, along with the headcount and average salary, sorted by average salary descending.

Answer: SELECT JobTitle, COUNT(*) AS Headcount, AVG(Salary) AS AvgSalary FROM EmployeeDemographics ED JOIN EmployeeSalary ES ON ED.EmployeeID = ES.EmployeeID GROUP BY JobTitle HAVING COUNT(*) > 2 ORDER BY AVG(Salary) DESC; — Explanation: GROUP BY partitions by JobTitle. COUNT(*) counts employees in each group. HAVING filters out groups with 2 or fewer employees. ORDER BY sorts the filtered result by average salary.

Q6. You need to drop a Customer table, but it is referenced by an Orders table via a foreign key. What steps must you take?

Answer: Step 1: Identify the FK constraint name: SELECT name FROM sys.foreign_keys WHERE referenced_object_id = OBJECT_ID('Customer'); Step 2: Either (a) Drop Orders first: DROP TABLE Orders; OR (b) Remove the FK constraint: ALTER TABLE Orders DROP CONSTRAINT FK_Orders_Customer; Step 3: Then drop Customer: DROP TABLE Customer; Alternatively, if you want to keep Orders but remove the FK dependency, use option (b) and then drop Customer.

Q7. Explain why creating too many indexes on a table can degrade database performance, even though indexes are designed to speed up queries.

Answer: Each index is a separate B-tree structure that must be maintained. On every INSERT, the new row must be added to every index on the table. On every UPDATE of an indexed column, the index entry must be located, removed, and re-inserted in the correct position. On every DELETE, the row's entry must be removed from all indexes. Therefore, a table with 20 indexes will have significantly slower INSERT/UPDATE/DELETE operations than one with 3 indexes. Indexes also consume disk storage and buffer pool memory. Best practice: index only high-selectivity columns that appear frequently in queries.

Q8. A salary table shows some employees with NULL salaries. Write queries using COUNT(*) and COUNT(Salary) to demonstrate the difference in results, and explain the output.

Answer: SELECT COUNT(*) AS AllRows, COUNT(Salary) AS NonNullSalaries FROM Employee; — If there are 100 rows total and 8 have NULL salary: COUNT(*) = 100 (counts all rows regardless of NULLs). COUNT(Salary) = 92 (ignores NULL values in the Salary column). This behaviour is critical in reporting — always choose the correct COUNT variant to avoid under- or over-counting.

Q9. Write a query to rank employees by salary within each department using ORDER BY and a window function. Show only those in rank 1 or 2.

Answer: WITH RankedEmployees AS (SELECT EmployeeID, DepartmentID, Salary, RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS SalaryRank FROM Employee) SELECT EmployeeID, DepartmentID, Salary, SalaryRank FROM RankedEmployees WHERE SalaryRank <= 2 ORDER BY DepartmentID, SalaryRank; — PARTITION BY DepartmentID restarts ranking within each department. ORDER BY Salary DESC ranks highest salary as 1. RANK() gives equal rank to ties and skips the next rank.

Q10. A CHECK constraint is defined as CHECK (StartDate < EndDate) on a Projects table. A developer tries to insert a row where StartDate = EndDate. What happens? How would you modify the constraint to allow equal dates?

Answer: The INSERT will fail with a constraint violation because the condition StartDate < EndDate evaluates to FALSE when StartDate = EndDate. To allow equal dates, change the constraint to: ALTER TABLE Projects DROP CONSTRAINT chk_project_dates; ALTER TABLE Projects ADD CONSTRAINT chk_project_dates CHECK (StartDate <= EndDate); Now equal start and end dates (same-day projects) are permitted.

Section D: Normalisation and Denormalisation (10 Questions)

Q1. A table OrderDetails has: OrderID, OrderDate, CustomerID, CustomerName, CustomerCity, ProductID, ProductName, ProductCategory, UnitPrice, Quantity. Normalise this table to 3NF step by step.

Answer: 1NF: Already atomic. Identify PK: (OrderID, ProductID). 2NF: Remove partial dependencies. CustomerID, CustomerName, CustomerCity depend only on OrderID (not full PK). ProductName, ProductCategory, UnitPrice depend only on ProductID. Result: Order(OrderID [PK], OrderDate, CustomerID [FK]), Customer(CustomerID [PK], CustomerName, CustomerCity), Product(ProductID [PK], ProductName, ProductCategory, UnitPrice), OrderLine(OrderID [PK,FK], ProductID [PK,FK], Quantity). 3NF: Check transitive dependencies. CustomerCity → depends on CustomerCity... actually CustomerCity might depend on a PostalCode if present. In this dataset, CustomerCity depends on CustomerID directly — no further transitive dependency. All tables are in 3NF.

Q2. What is a transitive dependency? Provide an example and show how it violates 3NF.

Answer: A transitive dependency exists when a non-key attribute determines another non-key attribute. Example: Employee(EmployeeID [PK], Name, DepartmentID, DepartmentName). Here, EmployeeID → DepartmentID (direct). DepartmentID → DepartmentName (transitive). So EmployeeID → DepartmentName transitively through DepartmentID. This violates 3NF. Fix: Employee(EmployeeID, Name, DepartmentID [FK]); Department(DepartmentID [PK], DepartmentName).

Q3. A social media platform stores user posts. For performance, the team decides to store the pre-computed like count in the Post table rather than counting from a Likes table. Is this normalisation or denormalisation? What risks does it introduce?

Answer: This is denormalisation — intentional introduction of redundancy (LikeCount) that could be derived from counting rows in the Likes table. Risks: (1) Inconsistency: if a like is inserted/deleted but LikeCount is not updated atomically, the count becomes stale. (2) Update anomaly: every like/unlike operation must update both the Likes table AND the Post.LikeCount. Mitigation: Use triggers or application-level logic (ideally within a transaction) to keep LikeCount consistent. For high-read, low-write scenarios, this is a valid and common optimisation.

Q4. Explain the difference between 2NF and BCNF. Under what circumstances is a table in 3NF but not in BCNF?

Answer: 2NF: No partial dependencies (every non-key attribute depends on the WHOLE primary key). BCNF: Every determinant must be a superkey (stronger than 3NF). A table can be in 3NF but not BCNF when there are multiple overlapping candidate keys. Example: Teaching(Student, Subject, Teacher) where: Each teacher teaches only one subject (Teacher → Subject). A student can have multiple teachers per subject (Student, Subject → Teacher). Candidate keys: (Student, Teacher) and (Student, Subject). Teacher → Subject, but Teacher is not a superkey. This is in 3NF (Subject is a prime attribute) but violates BCNF.

Q5. A developer argues that 'denormalisation makes databases better because it's faster.' Evaluate this claim.

Answer: This is partially true but oversimplified. Denormalisation improves READ performance by reducing JOINs and pre-computing results. However: (1) It degrades WRITE performance (INSERT/UPDATE/DELETE must maintain redundant data). (2) It risks data inconsistency if redundant copies fall out of sync. (3) It complicates application logic (more update statements required). Verdict: Denormalisation is appropriate for read-heavy analytics/reporting databases (OLAP) where data is loaded in bulk and rarely updated. It is generally harmful for OLTP databases where data integrity and frequent writes are paramount. Good design: normalise first, then selectively denormalise based on measured performance bottlenecks.

Q6. A hospital table Patient(PatientID, Name, DoctorID, DoctorName, DoctorSpecialty) stores patient-doctor assignments. Identify all normal form violations and normalise to 3NF.

Answer: 1NF: Satisfied (assuming one doctor per row). PK: PatientID. 2NF: DoctorID is a non-key attribute — no partial dependency issue (single-column PK). 3NF violation: PatientID → DoctorID, and DoctorID → DoctorName, DoctorID → DoctorSpecialty. Transitive dependency: PatientID → DoctorName and DoctorSpecialty via DoctorID. Fix (3NF): Patient(PatientID [PK], Name, DoctorID [FK]); Doctor(DoctorID [PK], DoctorName, DoctorSpecialty).

Q7. What does it mean for a table to be in 1NF, and why is it considered the foundation of normalisation?

Answer: A table is in 1NF when: (1) All column values are atomic (no arrays, lists, or nested structures). (2) Each column contains a single type of value. (3) All columns are named uniquely. (4) A primary key exists. It is foundational because: Without 1NF, higher normal forms cannot be evaluated or applied. Repeating groups and multi-valued attributes prevent the relational model from functioning correctly. SQL's set-based operations assume atomic, single-type cell values.

Q8. A retail company's Sales table currently uses JOINs across 5 tables for a dashboard query that runs 10,000 times per hour. The query is too slow. Propose a denormalisation strategy and discuss the trade-offs.

Answer: Strategy: Create a materialised/pre-joined summary table (e.g., SalesSummary) that combines the 5 tables' relevant columns. Populate it via a scheduled ETL job (e.g., every 15 minutes). Dashboard queries hit SalesSummary directly. Alternatively, add redundant columns (e.g., CustomerName, ProductName) directly to the Sales table to eliminate 2 of the 5 JOINs. Trade-offs: Benefit: Dramatically faster reads (no JOINs, optimised indexes). Cost: (1) Storage overhead for duplicate data. (2) Staleness: SalesSummary may be up to 15 minutes old. (3) ETL complexity. (4) Update anomalies if source data changes between refreshes.

Q9. Given Employee(EmpID, Name, Salary, DeptID, DeptName, ManagerID, ManagerName), identify all FDs (Functional Dependencies) and determine which normal form is violated.

Answer: Functional Dependencies: EmpID → Name, Salary, DeptID, ManagerID. DeptID → DeptName (transitive via EmpID). ManagerID → ManagerName (transitive via EmpID). 3NF Violation: DeptID → DeptName: non-key attribute (DeptID) determines another non-key attribute (DeptName) — transitive dependency. ManagerID → ManagerName: same issue. Fix: Employee(EmpID [PK], Name, Salary, DeptID [FK], ManagerID [FK]); Department(DeptID [PK], DeptName); Manager(ManagerID [PK], ManagerName) — or ManagerID FK → Employee(EmpID) for a self-referencing hierarchy.

Q10. A student suggests combining the Customer and Order tables into one table to avoid JOINs and improve performance. What problems would this create?

Answer: Problems: (1) Redundancy: CustomerName, CustomerAddress etc. would repeat for every order the customer places. (2) Update anomaly: changing a customer's address requires updating all their order rows. (3) Insertion anomaly: cannot store a new customer until they place an order. (4) Deletion anomaly: deleting a customer's only order deletes their record entirely. (5) The resulting table violates 2NF or 3NF depending on key definition. This is a classic case where premature denormalisation causes more harm than the JOIN cost. Use proper indexing on CustomerID in the Order table instead.

Section E: Transaction Management (10 Questions)

Q1. Explain the ACID property of Atomicity using a hotel room booking system as an example. What happens if the system crashes midway through the booking?

Answer: Atomicity means the transaction is all-or-nothing. In a hotel booking: BEGIN TRANSACTION → (1) Check room availability, (2) Create Booking record, (3) Deduct payment from guest account, (4) Send confirmation email record. If the system crashes after step 2 but before step 3, atomicity ensures the booking record is rolled back — the guest is not charged and no orphan booking exists. This is enforced via the transaction log: on restart, SQL Server's recovery process rolls back any uncommitted transactions using the WAL (Write-Ahead Log).

Q2. Two transactions T1 and T2 both read a bank account balance of $5,000. T1 withdraws $1,000 and T2 withdraws $2,000. If no concurrency control is in place, what problem occurs and what is the final balance?

Answer: This is a Lost Update problem. Both T1 and T2 read Balance=5000. T1 writes 5000-1000=4000. T2 writes 5000-2000=3000 (T2 overwrites T1's update because it used the old value it read). Final balance: $3,000 — but the correct balance should be $2,000 (5000-1000-2000). T1's withdrawal of $1,000 is effectively lost. Solution: Use proper locking (REPEATABLE READ or SERIALIZABLE isolation) or optimistic concurrency with version checks.

Q3. What is a deadlock in a database context? Describe a specific scenario involving two transactions and explain how SQL Server resolves it.

Answer: A deadlock occurs when T1 holds a lock that T2 needs, and T2 holds a lock that T1 needs, creating a circular wait. Scenario: T1 locks Table_A (X lock) and waits for Table_B. T2 locks Table_B (X lock) and waits for Table_A. Neither can proceed. SQL Server's deadlock monitor detects the cycle (typically within 5 seconds), chooses the transaction with the lower deadlock priority (or lower cost) as the 'victim,' kills it with error 1205, and rolls it back. The surviving transaction proceeds. The killed transaction should be retried by the application.

Q4. Compare READ COMMITTED and SERIALIZABLE isolation levels. When would you choose one over the other?

Answer: READ COMMITTED (default): Prevents dirty reads. Allows non-repeatable reads and phantom reads. Lower overhead; higher concurrency. Choose for most OLTP operations where slight read inconsistency is acceptable (e.g., displaying product listings, generating non-critical reports). SERIALIZABLE: Prevents all three phenomena (dirty, non-repeatable, phantom). Transactions execute as if fully sequential. Highest consistency but lowest concurrency; more locking overhead; higher deadlock risk. Choose for financial transactions, inventory reservation, or any scenario where phantom rows would cause incorrect business decisions (e.g., seat booking, stock allocation).

Q5. Write a T-SQL transaction with proper error handling (TRY-CATCH) that transfers funds between two bank accounts. The transfer should only commit if both the debit and credit succeed.

Answer: BEGIN TRY; BEGIN TRANSACTION; UPDATE BankAccount SET Balance = Balance - 500 WHERE AccountID = 101 AND Balance >= 500; IF @@ROWCOUNT = 0 THROW 50001, 'Insufficient funds.', 1; UPDATE BankAccount SET Balance = Balance + 500 WHERE AccountID = 202; COMMIT TRANSACTION; PRINT 'Transfer successful.'; END TRY BEGIN CATCH; IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; PRINT 'Error: ' + ERROR_MESSAGE(); END CATCH — The @@ROWCOUNT check ensures the debit row was actually updated (catches insufficient funds). If either UPDATE fails, the CATCH block rolls back both changes.

Q6. What is a dirty read? Give a scenario in a stock trading system where a dirty read could lead to a serious business error.

Answer: A dirty read occurs when Transaction T1 reads data modified by T2, which has not yet committed. If T2 later rolls back, T1 has acted on non-existent data. Scenario: T2 updates StockQuantity from 100 to 0 (to reserve stock for order). T1 reads StockQuantity = 0 at READ UNCOMMITTED. T1 rejects a customer's order ('out of stock'). T2 then fails and rolls back — StockQuantity returns to 100. Result: A valid order was incorrectly rejected, and stock was available the whole time. In trading systems, this could cause incorrect price quotes, failed legitimate orders, or erroneous margin calls.

Q7. Explain the role of transaction logs (Write-Ahead Logging) in database recovery after a system crash.

Answer: Write-Ahead Logging (WAL) requires that every change is first written to the transaction log before being applied to the actual data pages. On crash recovery: (1) SQL Server reads the transaction log. (2) REDO phase: re-applies all changes from committed transactions that were not yet flushed to disk (ensures durability). (3) UNDO phase: reverses all changes from transactions that were in-flight (uncommitted) at crash time (ensures atomicity). This guarantees the database returns to a consistent

state after any failure. The transaction log is the cornerstone of ACID's Atomicity and Durability guarantees.

Q8. A long-running report query is holding shared locks and blocking an urgent sales update. What isolation level change or technique would you apply to resolve this, and what are the trade-offs?

Answer: Solution 1: Change the report query to READ UNCOMMITTED (or add WITH(NOLOCK) hint): SELECT * FROM Sales WITH(NOLOCK). Allows the report to read without acquiring shared locks, unblocking the update. Trade-off: Report may see uncommitted or inconsistent data. Solution 2 (better): Enable SNAPSHOT isolation: ALTER DATABASE SalesDB SET ALLOW_SNAPSHOT_ISOLATION ON; SET TRANSACTION ISOLATION LEVEL SNAPSHOT. Report reads a consistent snapshot from row versioning (stored in tempdb). No shared locks held; update proceeds. Trade-off: Extra storage and I/O in tempdb; some overhead on write operations.

Q9. What is a SAVEPOINT, and in what scenario would using one be better than a simple COMMIT/ROLLBACK?

Answer: A SAVEPOINT marks an intermediate point within a transaction to which you can roll back without aborting the entire transaction. Scenario: A multi-step order processing transaction: (1) Create Order header. SAVE TRANSACTION sp_order. (2) Process each order line. If a specific line fails (e.g., product discontinued), ROLLBACK TO sp_order rolls back only the failed line processing, preserving the Order header. Alternative lines can be retried or skipped. Without savepoints, any failure would require rolling back the entire transaction (including the Order header), forcing a full restart.

Q10. Describe a scenario where the REPEATABLE READ isolation level prevents a problem that READ COMMITTED would not.

Answer: Scenario: An accounting system calculates a running total by reading the same account balance twice within a transaction. Under READ COMMITTED: T1 reads Balance = 10,000. Another transaction T2 updates Balance to 8,000 and commits. T1 reads Balance again = 8,000. T1 gets inconsistent data (non-repeatable read) — its two reads disagree, leading to an incorrect report. Under REPEATABLE READ: T1 holds a shared lock on the balance row after first read. T2 cannot modify it until T1 completes. T1's second read still returns 10,000. The report is consistent. REPEATABLE READ prevents non-repeatable reads; READ COMMITTED does not.

Section F: Data Backup and Recovery (10 Questions)

Q1. A company's production database uses the SIMPLE recovery model. A developer accidentally runs DELETE FROM Orders (without WHERE) at 2:00 PM. The last full backup was at midnight. What is the maximum data recovery possible, and what data is irrecoverable?

Answer: Under SIMPLE recovery model, transaction log backups are NOT taken — the log is auto-truncated at checkpoints. Maximum recovery: restore the full backup from midnight. Data recoverable: all data as of midnight. Data irrecoverable: 14 hours of transactions (midnight to 2:00 PM). No point-in-time recovery is possible. Lesson: For production OLTP databases, always use the FULL recovery model with transaction log backups to minimise data loss (RPO). In this case, switching to FULL recovery with hourly log backups would limit loss to at most 1 hour.

Q2. Explain the 3-2-1 backup rule and why it is considered a best practice. Give a scenario where violating it could cause total data loss.

Answer: 3-2-1 Rule: Keep 3 copies of data (original + 2 backups), on 2 different media types (e.g., SSD + tape/cloud), with 1 copy offsite (geographically separated). Violation scenario: A company stores all backups on the same NAS (Network Attached Storage) in the same server room as the production database. A fire destroys the server room. Both the primary database AND all backups are destroyed simultaneously. Total data loss results. The 3-2-1 rule specifically prevents this: the offsite copy survives even if the primary site is completely destroyed.

Q3. What is the difference between RPO and RTO? A healthcare database has RPO = 15 minutes and RTO = 1 hour. Design an appropriate backup strategy.

Answer: RPO (Recovery Point Objective): Maximum acceptable data loss. RPO = 15 min means at most 15 minutes of data can be lost. RTO (Recovery Time Objective): Maximum acceptable downtime. RTO = 1 hour means the system must be back online within 1 hour. Strategy: Recovery model: FULL. Full backup: Weekly (Sunday midnight). Differential backup: Daily (every day at midnight). Transaction log backup: Every 15 minutes (satisfies RPO = 15 min). For RTO = 1 hour: Pre-stage backup files on standby server; practice restore drills to ensure the full + differential + log restoration process completes within 1 hour. Consider: SQL Server Always On Availability Group with a standby replica for near-zero RTO.

Q4. Write the T-SQL commands to perform a full database backup, followed by a differential backup, and then a transaction log backup for a database named 'HospitalDB'.

Answer: -- Full backup: BACKUP DATABASE HospitalDB TO DISK = 'E:\Backups\HospitalDB_Full.bak' WITH INIT, NAME = 'HospitalDB Full', STATS = 10; -- Differential: BACKUP DATABASE HospitalDB TO DISK = 'E:\Backups\HospitalDB_Diff.bak' WITH DIFFERENTIAL, NAME = 'HospitalDB Diff'; -- Transaction Log: BACKUP LOG HospitalDB TO DISK = 'E:\Backups\HospitalDB_Log.bak' WITH NAME = 'HospitalDB Log'; Note: Differential backup requires the database to be in FULL or BULK_LOGGED recovery model. Log backup requires FULL recovery model.

Q5. What is DBCC CHECKDB and when should you run it? What does it do?

Answer: DBCC CHECKDB checks the logical and physical integrity of all objects in a SQL Server database. It performs: (1) Allocation checks: Ensures pages are correctly allocated in data files. (2) Structural integrity: Checks B-tree structure of tables and indexes. (3) Constraint validation: Validates FK, CHECK, and NOT NULL constraints. (4) Cross-object consistency: Ensures indexes match their base table data. When to run: Weekly for most production databases (during low-traffic windows); after major data imports or migrations; after a hardware failure or unexpected shutdown; as part of post-restore validation. Command: DBCC CHECKDB ('HospitalDB') WITH NO_INFOMSGS, ALL_ERRORMSGS;

Q6. A database restoration is needed after a disaster. You have: a Full backup from Monday, a Differential from Wednesday, and Transaction Log backups from every 2 hours from Monday to Friday. The failure occurred Friday at 3:45 PM. Describe the complete restoration sequence.

Answer: Step 1: RESTORE DATABASE ... FROM DISK = 'Monday_Full.bak' WITH NORECOVERY; Step 2: RESTORE DATABASE ... FROM DISK = 'Wednesday_Diff.bak' WITH NORECOVERY; Step 3: Apply each transaction log backup from Wednesday evening through Friday 3:30 PM (the last log before the failure), each WITH NORECOVERY. Step 4: RESTORE LOG ... FROM DISK = 'Friday_14:00_Log.bak' WITH STOPAT = '2025-01-10 15:44:59', RECOVERY; — This brings the database online at the last moment before the failure. The RECOVERY keyword on the final step takes the database out of restoring state.

Q7. What is the difference between FULL, SIMPLE, and BULK_LOGGED recovery models in SQL Server? Give a use case for each.

Answer: SIMPLE: Log auto-truncated at checkpoints; no log backups needed; only restore to last backup. Use case: Development/test databases; databases where data can be re-generated or reloaded. FULL: Log only truncated when a log backup is taken; full point-in-time recovery possible. Use case: Production OLTP systems (banking, ERP, healthcare) where minimal data loss is required. BULK_LOGGED: Like FULL, but bulk operations (BULK INSERT, SELECT INTO, index rebuilds) are minimally logged for performance. Point-in-time recovery is NOT available during bulk operations. Use case: ETL loading windows in data warehouses — use BULK_LOGGED during the load, then switch back to FULL.

Q8. Explain why 'testing backups by performing restore drills' is listed as a best practice. What risks exist with untested backups?

Answer: An untested backup may appear to complete successfully but could be: (1) Corrupted: The backup file is physically damaged (media failure, storage error). (2) Incomplete: Backup stopped early due to timeout or error but logged as success. (3) Incompatible: Backup from one SQL Server version cannot

be restored to a different version or edition. (4) Too slow to meet RTO: The restoration process may take longer than the recovery window allows. (5) Missing components: Filegroups, secondary data files, or certificates (for encrypted databases) may not have been included. Regular restore drills — on a separate test server — verify the backup is actually restorable, the process fits within RTO, and team members know the procedure under pressure.

Q9. A company performs weekly full backups on Sunday and stores them on the same server as the database. A ransomware attack encrypts all files on the server on Thursday. How should the backup strategy be improved?

Answer: Problems with current strategy: (1) Single copy on the same machine — backup and database encrypted simultaneously. (2) No offsite copy — no survivable copy exists. (3) Weekly-only — up to 7 days of data loss. Improved strategy: (1) Follow 3-2-1 rule: store backups on a separate dedicated backup server (different machine), plus cloud storage (Azure Blob, AWS S3) or tape offsite. (2) Enable immutable/WORM (Write Once Read Many) storage for cloud backups — ransomware cannot encrypt immutable buckets. (3) Increase frequency: daily differential + transaction log backups. (4) Restrict backup service account permissions so ransomware running as application user cannot delete backup files. (5) Test restores quarterly.

Q10. A junior DBA recommends using TRUNCATE TABLE to 'back up' old records before deleting them. Evaluate this suggestion.

Answer: This suggestion is fundamentally incorrect. TRUNCATE TABLE is not a backup — it removes all rows from the table immediately and irreversibly (with no row-by-row logging and no recovery path beyond a transaction rollback before commit). Correct approaches for preserving data before deletion: (1) INSERT INTO Archive_Table SELECT * FROM SourceTable WHERE condition; — copies records to an archive table before deletion. (2) SELECT * INTO BackupTable FROM SourceTable — creates a copy of the entire table. (3) Take a database backup before running the deletion. (4) Use a soft-delete pattern: add a 'DeletedAt' column and UPDATE instead of DELETE. TRUNCATE is used to efficiently empty tables when data is no longer needed, not as a preservation mechanism.

Summary Reference Card

Topic	Key Concepts	Critical SQL / Notation
ER Model	Entities, Attributes (simple/composite/multi-valued/derived), Keys, Cardinality (1:1, 1:N, M:N), Participation (total/partial)	CREATE TABLE with PK, FK, UNIQUE, NOT NULL
ISA / Aggregation	Generalisation (bottom-up), Specialisation (top-down), Inheritance, IS-A triangle, Disjoint vs Overlapping, Total vs Partial	Table-per-Hierarchy, Table-per-Subclass
Advanced SQL	INDEX types, COUNT(*) vs COUNT(col), ORDER BY + OFFSET-FETCH, HAVING vs WHERE, DROP TABLE/INDEX, CHECK constraint syntax	CREATE INDEX, GROUP BY...HAVING, CHECK (col condition)
Normalisation	1NF (atomic), 2NF (no partial deps), 3NF (no transitive deps), BCNF (every determinant = superkey), Denormalisation trade-offs	Decompose tables; enforce via FK + constraints
Transactions	ACID (Atomicity, Consistency, Isolation, Durability), Concurrency problems (dirty/phantom/lost update), Isolation levels, Deadlocks, Locking	BEGIN TRAN; COMMIT; ROLLBACK; TRY-CATCH; SAVEPOINT
Backup & Recovery	Full, Differential, Log backups; RPO & RTO; 3-2-1 rule; Recovery models (SIMPLE/FULL/BULK_LOGGED); DBCC CHECKDB	BACKUP DATABASE...WITH DIFFERENTIAL; RESTORE...WITH NORECOVERY; DBCC CHECKDB